BÁO CÁO DỰ ÁN

XÂY DỰNG HỆ THỐNG GIÁM SÁT VÀ PHÁT HIỆN BẤT THƯỜNG MẠNG TRÊN NỀN TẢNG PNETLAB

NETWORK MONITORING • INTRUSION DETECTION • ANOMALY ANALYSIS

NHÓM THỰC HIỆN

Trần Hoàng Minh Vương · Diệp Mạnh Tuấn

Đà Nẵng • 2026

MỤC LỤC

MỞ ĐẦU **7**
CHƯƠNG 1. BÀI TOÁN VÀ NỀN TẢNG KỸ THUẬT **8**
1.1 Bài toán giám sát mạng và an ninh 8
1.2 Thu thập và lưu trữ số liệu vận hành 9
1.3 Phát hiện xâm nhập từ lưu lượng mạng 11
1.4 Phát hiện bất thường từ dữ liệu lịch sử 13
1.5 Nền tảng ứng dụng và môi trường giả lập 14
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **17**
2.1 Yêu cầu chức năng và tiêu chí vận hành 17
2.2 Thiết kế topology và luồng dữ liệu 19
2.3 Kiến trúc phần mềm và vòng đời cảnh báo 21
2.4 Thiết kế các cơ chế phát hiện và phân tích 26
2.5 Định vị giải pháp và phạm vi so sánh 27
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG NETMON SOC **30**
3.1 Chuẩn bị môi trường giả lập 30
3.2 Thiết lập mạng lõi và dịch vụ thử nghiệm 33
3.3 Triển khai pipeline giám sát và IDS 35
3.4 Xây dựng backend, mô hình và cảnh báo 38
3.5 Giao diện và hành trình của người vận hành 41
CHƯƠNG 4. KIỂM THỬ, KẾT QUẢ VÀ PHÂN TÍCH **45**
4.1 Phương pháp và phạm vi kiểm thử 45
4.2 Kiểm thử giám sát tình trạng hạ tầng 46
4.3 Kiểm thử phát hiện xâm nhập theo vùng mạng 48
4.4 Kiểm thử học máy và vòng đời cảnh báo 57
4.5 Đánh giá kết quả và giới hạn số liệu 59
CHƯƠNG 5. ĐÁNH GIÁ TỔNG THỂ VÀ HƯỚNG PHÁT TRIỂN **62**
5.1 Kết quả chính và giá trị của dự án 62
5.2 Giới hạn kỹ thuật và vận hành 63
5.3 Lộ trình hoàn thiện 64
KẾT LUẬN **66**
TÀI LIỆU THAM KHẢO **67**

TÓM TẮT NỘI DUNG VÀ KẾT QUẢ DỰ ÁN

NetMon SOC là môi trường thử nghiệm giám sát mạng và an ninh, được phát triển nhằm đặt ba nguồn thông tin trên cùng một giao diện: số liệu vận hành, sự kiện xâm nhập và cảnh báo bất thường. Hệ thống không chỉ cho biết thiết bị đang hoạt động hay ngừng phản hồi, mà còn giúp người vận hành đối chiếu thời điểm, phân vùng mạng và mức độ ảnh hưởng của các tín hiệu quan sát được.

Mô hình triển khai trên PNETLab gồm sáu thiết bị mục tiêu: pfSense, một router Cisco IOSv, hai switch Layer 3 và hai máy chủ Ubuntu. Prometheus thu thập SNMP và Node Exporter; Suricata trên pfSense ghi sự kiện EVE JSON; NetMon SOC dùng Flask và PostgreSQL để tổ chức dữ liệu, Isolation Forest để phát hiện điểm ngoại lai và Telegram để chuyển thông báo. AI Agent sử dụng Groq API để hỗ trợ tra cứu ngữ cảnh, không tự thực thi biện pháp xử lý sự cố.

Nhóm thực hiện kiểm thử có kiểm soát theo ba lớp: giám sát trạng thái/tài nguyên, nhận diện sự kiện IDS trên các phân vùng mạng và xử lý vòng đời cảnh báo. Báo cáo đối chiếu 13 kịch bản chính với kết quả ghi nhận; các menu sinh lưu lượng thử nghiệm WAN, Web, FTP và DNS được trình bày như nhóm kiểm thử mở rộng. Mỗi kết luận được giới hạn theo điều kiện giả lập và loại bằng chứng hiện có, không thay thế phép đo độ chính xác trên dữ liệu sản xuất.

Kết quả cho thấy đường đi từ metrics hoặc sự kiện Suricata tới giao diện và thông báo có thể vận hành trong phạm vi lab. Giá trị của dự án là kiến trúc tích hợp có thể tái hiện và kiểm chứng từng bước; giới hạn chủ yếu gồm quy mô thử nghiệm nhỏ, dữ liệu học máy ngắn hạn, chưa đánh giá đầy đủ false positive bằng tập nhãn độc lập và chưa hoàn thiện xác thực người dùng cho giao diện web.

Từ khóa: NetMon SOC; PNETLab; Prometheus; SNMP; Suricata; Isolation Forest; Flask; PostgreSQL; Telegram; Groq.

THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Thuật ngữ	Ý nghĩa trong báo cáo
SOC	Security Operations Center — Nền tảng theo dõi và phân tích sự kiện an ninh.
NPM / NSM	Network Performance / Security Monitoring — Giám sát hiệu năng / an ninh mạng.
SNMP	Simple Network Management Protocol — Thu thập trạng thái và bộ đếm thiết bị.
OID / MIB	Object Identifier / Management Information Base — Định danh và cấu trúc dữ liệu SNMP.
TSDB / PromQL	Time-Series Database / Prometheus Query Language — Lưu và truy vấn dữ liệu theo thời gian.
IDS / IPS	Intrusion Detection / Prevention System — Phát hiện / ngăn chặn xâm nhập.
EVE JSON	Định dạng sự kiện JSON do Suricata xuất ra.
API / REST	Application Programming Interface / Representational State Transfer.
ML / LLM	Machine Learning / Large Language Model — Học máy / mô hình ngôn ngữ lớn.
DMZ / VLAN	Demilitarized Zone / Virtual LAN — Vùng cách ly dịch vụ / mạng cục bộ ảo.
OSPF	Open Shortest Path First — Giao thức định tuyến nội bộ.
MTTR	Mean Time to Repair — Thời gian trung bình khắc phục sự cố.

DANH MỤC HÌNH MINH HỌA

Hình 2.1. Sơ đồ topology mạng tổng thể hệ thống NetMon SOC 19
Hình 2.2. Luồng dữ liệu giám sát trong hệ thống NetMon SOC 21
Hình 2.3. Kiến trúc phân lớp của hệ thống NetMon SOC 22
Hình 2.4. Cấu trúc thư mục dự án 24
Hình 2.5. Vòng đời cảnh báo (Alert Lifecycle) trong hệ thống 26
Hình 3.1. Topology mạng trong PNETLab 32
Hình 3.2. Giao diện Dashboard giám sát thiết bị thời gian thực 42
Hình 3.3. Giao diện trang quản lý cảnh báo (Alerts Management) 42
Hình 3.4. Giao diện SOC Dashboard phân tích sự kiện Suricata IDS 43
Hình 3.5. Giao diện AI Agent với phân tích ngữ cảnh mạng thời gian thực 44
Hình 4.1. Thông báo Telegram khi pfSense offline và tự động giải quyết 47
Hình 4.2. Giao diện chính NetMon WAN Attack Console 50
Hình 4.3. Giao diện Web Attack Generator 52
Hình 4.4. Giao diện FTP Attack Menu 53
Hình 4.5. Giao diện DNS Attack Menu 54
Hình 4.6. Alert AI Anomaly phát hiện bất thường CPU và RAM trên Server02 58

DANH MỤC BẢNG BIỂU

Bảng 2.1. Quy hoạch địa chỉ IP theo phân vùng mạng 19
Bảng 2.2. Lược đồ bảng alerts - lưu trữ cảnh báo hệ thống 23
Bảng 2.3. Lược đồ bảng soc_events - lưu sự kiện Suricata IDS 23
Bảng 2.4. So sánh NetMon SOC với các giải pháp hiện có 28
Bảng 3.1. Phiên bản phần mềm sử dụng trong hệ thống 30
Bảng 3.2. Danh sách thiết bị trong hệ thống và vai trò chi tiết 30
Bảng 4.1. Tổng hợp các kịch bản kiểm thử 45
Bảng 4.2. Kết quả kiểm thử ngưỡng tài nguyên server 48
Bảng 4.3. Tổng hợp kết quả kiểm thử Suricata IDS sau khi mở rộng 55
Bảng 4.4. Kết quả kiểm thử vòng đời alert 59
Bảng 4.5. Tổng hợp kết quả toàn bộ kịch bản kiểm thử 59
Bảng 4.6. Độ trễ phát hiện của từng loại cảnh báo 60

MỞ ĐẦU

Các công cụ giám sát thiết bị thường trả lời câu hỏi “hạ tầng có hoạt động không?”, trong khi công cụ phân tích an ninh tìm dấu hiệu truy cập và lưu lượng đáng ngờ. Khi hai nguồn thông tin tách biệt, việc liên hệ một cảnh báo hiệu năng với một sự kiện bảo mật trở nên khó khăn. NetMon SOC được xây dựng để đặt dữ liệu vận hành, nhật ký Suricata và cảnh báo học máy vào cùng một quy trình quan sát, từ đó hỗ trợ kỹ thuật viên khoanh vùng sự cố trước khi đưa ra quyết định xử lý.

Nhóm xây dựng topology doanh nghiệp thu gọn trên PNETLab, triển khai thu thập SNMP và Node Exporter, tích hợp IDS, thiết kế quy tắc cảnh báo và thực hiện các phép thử có kiểm soát. Phương pháp đánh giá theo trình tự: xác định trạng thái ban đầu, tạo tác động, thu thập bằng chứng từ nguồn dữ liệu, đối chiếu trạng thái hiển thị và kiểm tra thông báo. Tài liệu chỉ phản ánh môi trường phòng thí nghiệm; chưa đo khả năng chịu tải ở quy mô lớn hoặc bảo đảm SLA.

CHƯƠNG 1. BÀI TOÁN VÀ NỀN TẢNG KỸ THUẬT

1.1 Bài toán giám sát mạng và an ninh

1.1.1 Vai trò của giám sát chủ động

Giám sát mạng (Network Monitoring) là quá trình thu thập và đánh giá trạng thái hạ tầng theo thời gian nhằm phát hiện mất kết nối, suy giảm hiệu năng hoặc hành vi cần điều tra. Đối tượng quan sát gồm router, switch, firewall, máy chủ và dịch vụ. Dữ liệu có ích khi đi kèm nguồn thu thập, mốc thời gian và ngữ cảnh thiết bị; một giá trị đo đơn lẻ không đủ xác định nguyên nhân sự cố.

Hai nhóm dữ liệu được sử dụng trong dự án là Network Performance Monitoring (NPM), theo dõi khả năng sẵn sàng, lưu lượng và tài nguyên; và Network Security Monitoring (NSM), ghi nhận sự kiện đáng ngờ từ lưu lượng và nhật ký. NetMon SOC không thay thế chuyên môn phân tích sự cố mà cung cấp một nơi đối chiếu tín hiệu từ các nguồn khác nhau.

Trong vận hành, một hệ thống giám sát chủ động cần đồng thời hỗ trợ phát hiện sớm, nhận biết xu hướng và giảm thời gian xác định nguyên nhân. Các giá trị như độ trễ, tình trạng interface hoặc CPU chỉ thực sự hữu ích khi có ngưỡng, lịch sử và hành động tiếp theo được xác định rõ. Nhật ký an ninh bổ sung bối cảnh cho quá trình điều tra, nhưng không tự chứng minh việc xâm nhập đã thành công.

Khó khăn của giám sát hiện đại không chỉ nằm ở số lượng thiết bị. Các sản phẩm khác nhau xuất metrics theo những giao thức và chu kỳ khác nhau; log sự kiện có cấu trúc, độ chi tiết và độ tin cậy khác nhau. Vì vậy, cần thống nhất thời gian ghi nhận, định danh thiết bị và quy tắc lưu trữ trước khi đưa các nguồn dữ liệu lên cùng dashboard.

Cảnh báo sai (false positive) và cảnh báo bị bỏ sót (false negative) là hai rủi ro đối nghịch. Ngưỡng quá nhạy làm người vận hành bị quá tải thông báo; ngưỡng quá lỏng có thể che khuất sự cố thực. Cơ chế debounce, nhắc lại có kiểm soát và đối chiếu nhiều nguồn chỉ giúp quản lý nhiễu; muốn lượng hóa chất lượng phát hiện phải có tập dữ liệu được gắn nhãn và tiêu chí đánh giá riêng.

1.1.2 Những yêu cầu đặt ra với NetMon SOC

Bài toán được cụ thể hóa thành ba câu hỏi: thiết bị hoặc dịch vụ nào đang suy giảm, bất thường xuất hiện từ lúc nào, và tín hiệu nào hỗ trợ khoanh vùng? Hệ thống cần phân biệt trạng thái thiết bị, trạng thái dịch vụ và dấu hiệu an ninh: máy chủ vẫn trả lời ICMP không có nghĩa HTTP đang sẵn sàng; lưu lượng tăng đột biến cũng không mặc nhiên là tấn công.

Để hỗ trợ người vận hành, NetMon SOC liên kết tên thiết bị, phân vùng mạng, chỉ số tài nguyên, sự kiện Suricata và thời điểm xuất hiện. Trong trường hợp CPU tăng cao cùng một cảnh báo IDS, hệ thống giúp xem hai tín hiệu cạnh nhau; việc xác định quan hệ nhân quả vẫn phải dựa vào kiểm tra tiến trình, lưu lượng và nhật ký tại điểm phát sinh.

Ba đầu ra cốt lõi là dashboard thể hiện trạng thái hiện tại, dữ liệu lịch sử phục vụ truy vấn và cơ chế thông báo đến người phụ trách. Tự động chặn địa chỉ IP, cô lập endpoint hoặc khôi phục cấu hình không thuộc phần đã thực nghiệm; những năng lực phản ứng tự động sẽ được đánh giá như hướng phát triển chứ không coi là chức năng hiện tại.

1.2 Thu thập và lưu trữ số liệu vận hành

1.2.1 SNMP và thông tin thiết bị mạng

SNMP (Simple Network Management Protocol) là giao thức tầng ứng dụng được định nghĩa trong RFC 1157 (SNMPv1), RFC 1901 (SNMPv2c) và RFC 3411-3418 (SNMPv3), được sử dụng rộng rãi để quản lý và giám sát thiết bị mạng. SNMP hoạt động theo mô hình Manager-Agent: Manager (máy chủ giám sát) gửi yêu cầu đến Agent (thiết bị cần giám sát) qua cổng UDP 161, Agent đọc giá trị từ cơ sở thông tin quản lý MIB (Management Information Base) và trả về kết quả.

Cấu trúc dữ liệu trong SNMP được tổ chức theo cây OID (Object Identifier) phân cấp. Ví dụ, OID 1.3.6.1.2.1.2.2.1.8 (ifOperStatus) biểu diễn trạng thái hoạt động của interface mạng: giá trị 1 là up, 2 là down, 3 là testing. Trong dự án này, module IF-MIB được sử dụng để thu thập các chỉ số giao diện mạng:

ifOperStatus: trạng thái hoạt động của interface

ifAdminStatus: trạng thái quản trị (bật/tắt bởi người dùng)

ifHCInOctets / ifHCOutOctets: số byte nhận/gửi tích lũy

ifHighSpeed: tốc độ cổng (Mbps)

ifAlias: tên mô tả interface do admin đặt

SNMPv2c được chọn trong lab nhờ khả năng tương thích với pfSense và Cisco IOSv, đồng thời cung cấp bộ đếm giao diện 64 bit. Tuy nhiên, SNMPv2c dùng community string không mã hóa; khi mở rộng ngoài môi trường cô lập cần ưu tiên SNMPv3, giới hạn nguồn truy cập và bảo vệ dữ liệu quản trị.

1.2.2 Node Exporter và dữ liệu máy chủ Linux

Node Exporter là thành phần của hệ sinh thái Prometheus, chạy dưới dạng daemon trên máy chủ Linux và xuất (export) hàng trăm chỉ số hệ thống qua giao diện HTTP trên cổng 9100. Prometheus định kỳ scrape endpoint /metrics của Node Exporter để thu thập dữ liệu.

node_cpu_seconds_total{mode="idle"}: thời gian CPU ở chế độ idle, dùng để tính tỷ lệ sử dụng CPU theo công thức: CPU% = 100 - (avg(irate(idle[5m])) * 100)

node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes: bộ nhớ khả dụng và tổng, dùng tính Memory Used %

node_filesystem_size_bytes / node_filesystem_avail_bytes: dung lượng và không gian còn trống của filesystem

node_network_receive_bytes_total / node_network_transmit_bytes_total: lưu lượng mạng theo từng interface

Node Exporter bổ sung những chỉ số hệ điều hành mà mô hình SNMP trên thiết bị mạng không phản ánh trực tiếp, chẳng hạn mức sử dụng CPU, bộ nhớ khả dụng và dung lượng filesystem. Hai nguồn không nên được đánh giá “nguồn nào chính xác hơn” một cách tuyệt đối: mỗi nguồn phục vụ một lớp thiết bị và có điều kiện thu thập riêng.

1.2.3 Prometheus, chuỗi thời gian và truy vấn

Prometheus là hệ thống giám sát và cơ sở dữ liệu chuỗi thời gian (time-series database) mã nguồn mở, được phát triển bởi SoundCloud và hiện thuộc hệ sinh thái Cloud Native Computing Foundation (CNCF). Prometheus áp dụng mô hình pull-based: máy chủ Prometheus chủ động gọi (scrape) các endpoint metrics theo chu kỳ cấu hình, thay vì nhận dữ liệu đẩy từ agent.

Mô hình dữ liệu của Prometheus là metric có nhãn (labeled metrics): mỗi time series được định danh bởi tên metric và tập hợp các nhãn key-value. Ví dụ: ifOperStatus{job="snmp-pfsense", instance="172.24.0.115", ifIndex="1"}. Ngôn ngữ truy vấn PromQL (Prometheus Query Language) hỗ trợ các phép toán vector, aggregation và range function mạnh mẽ.

Trong hệ thống NetMon SOC, Prometheus được cấu hình với chu kỳ scrape 15 giây (scrape_interval: 15s), quản lý 6 job tương ứng 6 thiết bị: 4 job SNMP (snmp-pfsense, snmp-core-router, snmp-swl3a, snmp-swl3b) và 2 job Node Exporter (node-server01, node-server02). Dữ liệu được lưu trữ trong storage nội bộ của Prometheus và truy vấn qua HTTP API tại endpoint /api/v1/query và /api/v1/query_range.

Chu kỳ lấy mẫu 15 giây tạo ra dữ liệu gần thời gian thực, không phải quan sát liên tục. Một sự kiện ngắn hơn khoảng cách giữa hai lần scrape có thể không tạo thành mẫu đo. Vì vậy, Suricata EVE JSON và các phép kiểm tra dịch vụ được dùng như nguồn bằng chứng bổ sung; dashboard cũng cần thể hiện rõ thời điểm cập nhật gần nhất.

1.3 Phát hiện xâm nhập từ lưu lượng mạng

1.3.1 IDS và cơ chế phát hiện của Suricata

Hệ thống phát hiện xâm nhập (Intrusion Detection System - IDS) là công cụ giám sát lưu lượng mạng hoặc hoạt động hệ thống nhằm phát hiện các hành vi vi phạm chính sách bảo mật, tấn công mạng hoặc sử dụng trái phép. IDS phân loại theo vị trí triển khai gồm: Network IDS (NIDS) giám sát lưu lượng mạng, Host IDS (HIDS) giám sát hệ thống cục bộ. Theo phương pháp phát hiện, IDS phân thành: Signature-based IDS (phát hiện dựa trên chữ ký tấn công đã biết) và Anomaly-based IDS (phát hiện dựa trên lệch chuẩn so với hành vi bình thường).

Khác với Intrusion Prevention System (IPS), IDS chỉ ghi nhận và cảnh báo mà không tự động ngăn chặn lưu lượng. Tuy nhiên, nhiều giải pháp hiện đại như Suricata hỗ trợ cả chế độ IDS lẫn IPS (inline mode).

Suricata là NIDS/NIPS mã nguồn mở hiệu năng cao, được phát triển bởi Open Information Security Foundation (OISF). Suricata hỗ trợ kiến trúc đa luồng (multi-threaded), tận dụng toàn bộ core CPU để xử lý song song gói tin, giúp đạt hiệu năng đáng chú ý khi so với Snort trong môi trường băng thông cao.

Luồng xử lý của Suricata bao gồm các bước: Capture (bắt gói tin từ card mạng qua AF_PACKET hoặc PCAP) → Decode (giải mã các tầng giao thức: Ethernet, IP, TCP/UDP, HTTP, DNS, TLS) → Detect (đối chiếu với tập luật Snort/Suricata rules) → Log (ghi kết quả ra file log). Suricata hỗ trợ nhiều định dạng output, trong đó quan trọng nhất là EVE JSON (Extensible Event Format) - định dạng JSON one-event-per-line ghi lại toàn bộ thông tin về alert, lưu lượng DNS, HTTP, TLS và các giao thức khác.

Trong dự án, Suricata được cài đặt trực tiếp trên pfSense và giám sát 5 interface độc lập: WAN (vtnet125952), Web-DMZ (vtnet33887), DNS-FTP (vtnet450526), LAN (vtnet210674) và ZeroTier (vtnet062106). Mỗi interface có file eve.json riêng, lưu tại /var/log/suricata/suricata_<interface>/eve.json.

1.3.2 EVE JSON và dữ liệu phục vụ phân tích SOC

EVE JSON là định dạng log chuẩn của Suricata, trong đó mỗi dòng là một đối tượng JSON hoàn chỉnh đại diện cho một sự kiện. Cấu trúc của một alert event điển hình bao gồm các trường: timestamp (thời điểm xảy ra, ví dụ 2025-04-27T13:20:00.123456+0700), event_type (loại sự kiện: alert, dns, http, tls), src_ip / src_port / dest_ip / dest_port (thông tin nguồn-đích), proto (giao thức TCP/UDP), alert.signature (tên luật kích hoạt), alert.category (phân loại tấn công như "Web Application Attack"), alert.severity (mức độ nghiêm trọng 1-4), alert.gid và alert.signature_id (định danh luật).

Hệ thống NetMon SOC ánh xạ alert.severity sang mức độ nội bộ theo bảng: Priority 1 (Critical), Priority 2 (Warning), Priority 3 (Info), Priority 4 (Info). Các danh mục Suricata cũng được chuẩn hóa sang danh mục ngắn gọn hơn, ví dụ "Web Application Attack" thành "Web Attack", "Detection of a Network Scan" thành "Port Scan".

Một bản ghi EVE JSON có thể mô tả lưu lượng không mang tính tấn công, còn trường `alert` chỉ xuất hiện khi có luật phù hợp. Trong thiết kế này, dữ liệu cần được chuẩn hóa trước khi hiển thị: timestamp để sắp xếp sự kiện, zone/interface để xác định vị trí quan sát, cặp địa chỉ nguồn–đích để nhận diện đối tượng và thông tin signature để giải thích lý do cảnh báo. Việc giữ lại mã luật giúp truy vết từ một dòng trên Dashboard về quy tắc gốc của Suricata.

1.4 Phát hiện bất thường từ dữ liệu lịch sử

1.4.1 Nguyên lý của Isolation Forest

Phát hiện bất thường (Anomaly Detection) là bài toán xác định các điểm dữ liệu khác biệt đáng kể so với phần còn lại của tập dữ liệu. Trong bối cảnh giám sát mạng, bất thường có thể là spike đột ngột trong lưu lượng (dấu hiệu tấn công DDoS), tăng đột biến CPU (malware đào coin), hay thay đổi bất thường trong traffic pattern (data exfiltration).

Isolation Forest (Liu, Ting & Zhou, 2008) là thuật toán phát hiện điểm ngoại lai dựa trên cấu trúc cây quyết định ngẫu nhiên. Ý tưởng cốt lõi: điểm dữ liệu bất thường (outlier) dễ bị "cô lập" hơn so với điểm bình thường trong không gian đặc trưng. Cụ thể, thuật toán xây dựng một tập hợp các cây phân tách (Isolation Trees): mỗi cây được tạo bằng cách chọn ngẫu nhiên một đặc trưng và một điểm phân tách, đệ quy chia nhỏ không gian dữ liệu cho đến khi mỗi điểm bị cô lập hoàn toàn.

Độ sâu trung bình của một điểm trong rừng cây tỷ lệ nghịch với mức độ bất thường: điểm bình thường nằm sâu trong cây (cần nhiều phân tách hơn để cô lập), điểm bất thường nằm nông (cô lập nhanh hơn). Anomaly score được tính theo công thức: s(x, n) = 2^(-E(h(x)) / c(n))

Trong đó E(h(x)) là chiều cao trung bình của điểm x trong rừng, c(n) là chiều cao trung bình trong Binary Search Tree với n điểm. Score tiến về 1 khi điểm là outlier rõ ràng, tiến về 0 khi là inlier bình thường.

1.4.2 Lựa chọn chỉ số và dữ liệu huấn luyện

Trong hệ thống NetMon SOC, Isolation Forest được ứng dụng để phát hiện bất thường trên 7 chỉ số (metrics) được phân theo loại thiết bị:

Đối với thiết bị mạng (firewall, router, switch): traffic_in_bps (lưu lượng vào tính bằng bit/giây), traffic_out_bps (lưu lượng ra), in_errors (lỗi đầu vào), out_errors (lỗi đầu ra).

Đối với máy chủ Linux: cpu_percent (tỷ lệ sử dụng CPU), mem_used_pct (tỷ lệ sử dụng bộ nhớ), disk_used_pct (tỷ lệ sử dụng ổ đĩa).

Mỗi metric được huấn luyện thành một file mô hình.pkl riêng biệt, lưu cùng với bộ chuẩn hóa StandardScaler để đảm bảo tính nhất quán khi dự đoán. Quá trình huấn luyện sử dụng dữ liệu lịch sử 7 ngày (168 giờ) từ Prometheus với bước lấy mẫu 5 phút, tạo ra khoảng 2016 điểm dữ liệu cho mỗi metric. Tham số contamination=0.05 (5% outlier) được thiết lập dựa trên giả định môi trường mạng bình thường không có quá 5% sự kiện bất thường. Cấu hình n_estimators=200 với random_state=42 đảm bảo tính tái lặp của kết quả.

Dữ liệu lịch sử khoảng bảy ngày, lấy mẫu theo bước năm phút, được dùng xây dựng baseline cho từng metric trong mô hình thực nghiệm. Tham số contamination=0.05 điều chỉnh giả định về tỷ lệ ngoại lai trong quá trình huấn luyện, không phải kết quả đo tỷ lệ cảnh báo sai. Các ngưỡng điểm bất thường cần được kiểm chứng trên dữ liệu độc lập trước khi đưa ra nhận định về độ chính xác.

1.5 Nền tảng ứng dụng và môi trường giả lập

1.5.1 Flask, REST API và PostgreSQL

Flask là micro web framework viết bằng Python, nổi tiếng với tính linh hoạt và đơn giản. Flask không ép buộc kiến trúc cụ thể nào, cho phép nhà phát triển tự do tổ chức mã nguồn theo cách phù hợp với bài toán. Trong hệ thống NetMon SOC, Flask đóng vai trò kép: vừa là HTTP server phục vụ giao diện web (server-side rendering với Jinja2 templates), vừa là REST API backend cung cấp dữ liệu cho frontend JavaScript.

Các endpoint REST API được tổ chức theo Blueprint pattern, phân chia thành 4 module: dashboard (giám sát thiết bị), alerts (quản lý cảnh báo), soc (phân tích Suricata), ai_agent (giao tiếp LLM). Các API tuân theo quy ước RESTful: GET để truy vấn dữ liệu, POST để thực hiện hành động. Mọi response trả về định dạng JSON chuẩn.

PostgreSQL được chọn làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) chính do khả năng xử lý tốt các truy vấn phức tạp với nhiều điều kiện lọc, hỗ trợ index composite hiệu quả và tính ổn định cao trong môi trường vận hành thực tế. SQLAlchemy ORM (Object-Relational Mapper) được sử dụng làm lớp trừu tượng, cô lập code ứng dụng khỏi chi tiết triển khai SQL cụ thể.

Cơ sở dữ liệu NetMon SOC bao gồm 3 bảng chính: alerts (lưu cảnh báo hệ thống), soc_events (lưu sự kiện Suricata), service_checks (lưu kết quả kiểm tra endpoint). Mỗi bảng được tối ưu hóa với các composite index phù hợp với pattern truy vấn thực tế.

Kết nối cơ sở dữ liệu sử dụng SQLAlchemy Engine với connection pool có kích thước DB_POOL_SIZE=10,DB_MAX_OVERFLOW=20, DB_POOL_TIMEOUT=30 giây, đảm bảo xử lý đồng thời hiệu quả từ các background thread.

1.5.2 Thông báo Telegram và hỗ trợ từ Groq LLM

Hệ thống thông báo được thiết kế theo kiến trúc mở rộng với lớp trừu tượng NotificationChannel (abstract base class). Mỗi kênh cụ thể (TelegramChannel, EmailChannel - placeholder) phải triển khai phương thức send(recipient_config, message). Lớp NotificationRouter chịu trách nhiệm định tuyến thông báo: xác định thành viên cần nhận, lọc theo loại cảnh báo và mức độ nghiêm trọng, sau đó gửi qua đúng kênh của từng người.

Telegram Bot API được sử dụng như kênh thông báo chính, gửi tin nhắn HTML-formatted qua endpoint https://api.telegram.org/bot{token}/sendMessage. Mỗi thành viên trong hệ thống có chat_id riêng, lưu trong biến môi trường để đảm bảo bảo mật.

Hệ thống hỗ trợ phân quyền thông báo theo vai trò: leader nhận tất cả loại cảnh báo (critical, warning, ai-anomaly, suricata, resolved, daily-summary, service-down); member chỉ nhận các cảnh báo liên quan đến thiết bị mình phụ trách và một số loại quan trọng (critical, warning, service-down, suricata). Cơ chế này giảm thiểu alert fatigue cho thành viên và đảm bảo thông tin quan trọng luôn đến đúng người.

AI Agent trong hệ thống hoạt động theo cơ chế Context-Augmented Generation: trước mỗi câu hỏi, hệ thống thu thập metrics thực tế từ Prometheus (trạng thái thiết bị,

lưu lượng, tài nguyên server), cùng với thông tin cảnh báo gần đây từ PostgreSQL, và đưa vào context window của LLM. System prompt được định nghĩa cụ thể với thông tin hạ tầng, quy tắc trả lời (chỉ dựa trên dữ liệu thực, không tự bịa đặt, gợi ý lệnh CLI khi cần) và ngôn ngữ phản hồi (tiếng Việt hoặc tiếng Anh theo ngôn ngữ của người dùng). Lịch sử hội thoại được giữ in-memory tối đa 16 lượt để duy trì ngữ cảnh liên tục.

1.5.3 PNETLab, ZeroTier và pfSense

PNETLab là môi trường giả lập mạng cho phép chạy đồng thời pfSense, router/switch Cisco IOSv và máy chủ Linux trên máy chủ ảo hóa. Nhóm sử dụng PNETLab để kiểm soát topology, thay đổi trạng thái thiết bị và lặp lại từng phép thử mà không tác động hạ tầng thật.

Trong topology của dự án, ZeroTier được cấu hình trên pfSense để tạo kết nối overlay từ máy quản trị (host OS) vào mạng lab PNETLab. Điều này cho phép máy quản trị bên ngoài truy cập trực tiếp vào tất cả thiết bị trong lab (10.20.0.0/16) thông qua địa chỉ ZeroTier của pfSense (172.24.0.115). Suricata cũng giám sát interface ZeroTier để phát hiện bất thường từ kết nối quản trị.

pfSense là nền tảng tường lửa/gateway trên FreeBSD, cung cấp NAT, chính sách truy cập và khả năng cài thêm gói IDS. Trong dự án, việc đặt Suricata trên pfSense giúp quan sát lưu lượng tại nhiều phân vùng từ một điểm chung.

Trong NetMon SOC, pfSense đồng thời là gateway/NAT, SNMP Agent, nơi chạy Suricata và điểm kết nối ZeroTier. Cách bố trí này giúp lab gọn nhưng tạo điểm phụ thuộc chung: sự cố pfSense có thể ảnh hưởng nhiều lớp giám sát. Khi áp dụng thực tế, cần xem xét tách tải và dự phòng các chức năng quan trọng.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Yêu cầu chức năng và tiêu chí vận hành

2.1.1 Yêu cầu giám sát, phát hiện và cảnh báo

Hệ thống cần theo dõi sáu thiết bị mục tiêu thông qua bốn job SNMP và hai job Node Exporter, hiển thị trạng thái liên kết, interface, lưu lượng và tài nguyên máy chủ. Chu kỳ scrape được cấu hình ở mức 15 giây. Với dịch vụ Web, TCP và DNS, các kiểm tra endpoint được đề xuất theo chu kỳ 30 giây để bổ sung thông tin mà trạng thái thiết bị không phản ánh.

Nhóm chức năng giám sát thiết bị (Device Monitoring): Hệ thống cần thu thập metrics từ thiết bị mạng (SNMP) và máy chủ Linux (Node Exporter) với chu kỳ 15 giây. Với mỗi thiết bị, cần theo dõi: trạng thái kết nối (online/offline), trạng thái từng interface (up/down), lưu lượng vào/ra trên mỗi interface, tỷ lệ lỗi (error rate), thời gian hoạt động (uptime). Riêng với máy chủ Linux, bổ sung các chỉ số CPU%, RAM%, Disk%. Giao diện cần hiển thị dữ liệu gần thời gian thực và vẽ đồ thị lịch sử cho 60 phút gần nhất.

Nhóm chức năng kiểm tra dịch vụ (Service Health Check): Hệ thống cần định kỳ (30 giây) kiểm tra trạng thái của các endpoint dịch vụ qua các giao thức: HTTP/HTTPS (đo status code và latency), TCP (kết nối cổng), DNS (phân giải tên miền). Kết quả được lưu vào cơ sở dữ liệu và hiển thị trên dashboard.

Nhóm chức năng phát hiện xâm nhập (IDS Integration): Hệ thống cần đọc log EVE JSON từ Suricata qua SSH mỗi 10 giây, parse và lưu các alert vào cơ sở dữ liệu. Giao diện SOC cần hiển thị: tổng số sự kiện theo mức độ nghiêm trọng, phân bố theo giao thức và phân vùng mạng, top IP nguồn tấn công, timeline sự kiện theo giờ, và bảng chi tiết sự kiện có khả năng lọc/tìm kiếm.

Cơ chế cảnh báo được thiết kế theo ngưỡng CPU > 85%, RAM > 90%, Disk > 85%, lưu lượng > 800 Mbps, tốc độ lỗi > 10/s hoặc interface mất hoạt động. Để giảm nhiễu, một số loại sự cố yêu cầu ba lần quan sát lỗi liên tiếp trước khi xác nhận; sau khi điều kiện lỗi biến mất, hệ thống áp dụng thời gian chờ trước khi tự giải quyết. Giá trị cụ thể của từng điều kiện cần đối chiếu với mã cấu hình đang chạy.

Nhóm chức năng AI Agent: Cung cấp giao diện chat cho phép operator đặt câu hỏi về trạng thái mạng bằng ngôn ngữ tự nhiên (tiếng Việt hoặc tiếng Anh). AI phải trả lời dựa trên dữ liệu metrics thực tế, không tự bịa thông tin, và gợi ý lệnh CLI cụ thể khi cần.

2.1.2 Yêu cầu hiệu năng, an toàn và khả năng bảo trì

Về hiệu năng, mục tiêu thiết kế là giao diện phản hồi trong khoảng hai giây và cập nhật metrics theo chu kỳ 15 giây. Đây là tiêu chí nghiệm thu dự kiến; kết quả đo ở Chương 4 chỉ phản ánh nhóm API và điều kiện thử đã ghi nhận, chưa tương đương chứng nhận hiệu năng cho mọi truy vấn hoặc số lượng người dùng.

Về tính ổn định, ứng dụng dùng cache TTL 12 giây nhằm giảm truy vấn lặp đến Prometheus và PostgreSQL connection pool với các thông số 10 kết nối cơ sở, tối đa 20 kết nối tràn. Cơ chế xử lý lỗi ở mỗi background worker giúp giới hạn ảnh hưởng của một thao tác thất bại, nhưng độ sẵn sàng dài hạn cần thêm phép thử tải và gián đoạn dịch vụ.

Bảo trì: Toàn bộ cấu hình tập trung trong file config.py duy nhất - thêm thiết bị mới, thêm thành viên, điều chỉnh ngưỡng chỉ cần sửa một file. Kiến trúc Repository Pattern tách biệt logic truy vấn khỏi business logic, dễ thay thế cơ sở dữ liệu nếu cần.

Bảo mật: Toàn bộ thông tin nhạy cảm (token Telegram, API key Groq, thông tin SSH) lưu trong biến môi trường (.env), không hard-code trong mã nguồn. SSH key-based authentication cho kết nối đến pfSense.

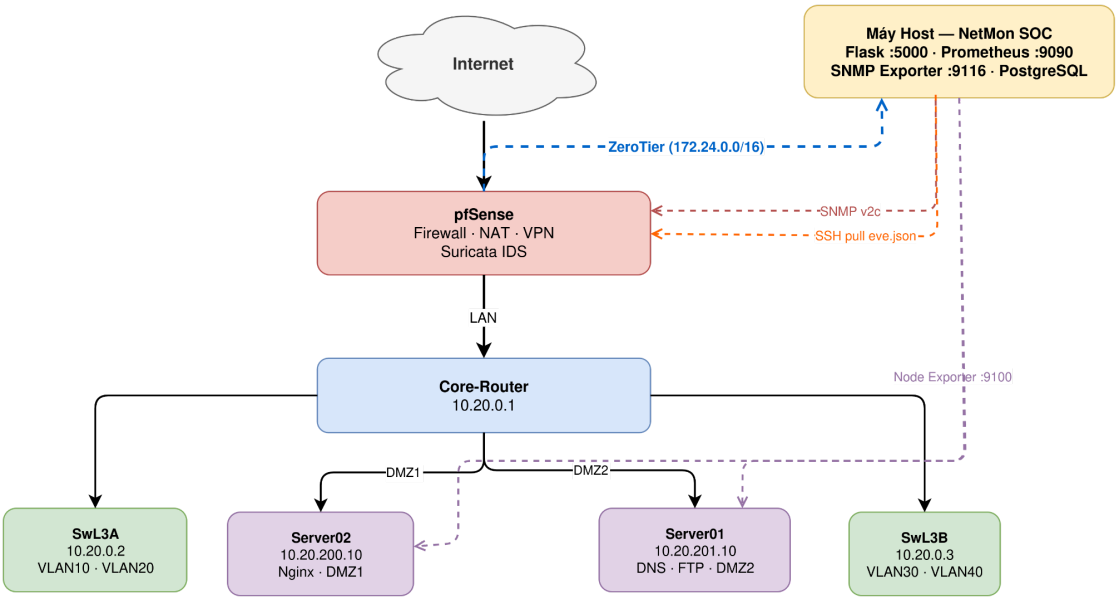
Khả năng mở rộng: Kiến trúc hỗ trợ thêm thiết bị giám sát mới chỉ bằng cách append entry vào danh sách DEVICES trong config.py, không cần sửa code. Kênh thông báo mới (Email, Slack) có thể thêm bằng cách kế thừa class NotificationChannel và đăng ký vào registry.

Đưa token Telegram, khóa Groq và thông tin SSH vào biến môi trường là bước quản lý bí mật cần thiết nhưng chưa đủ để bảo vệ ứng dụng. Bản thử nghiệm chưa có cơ chế đăng nhập và phân quyền đầy đủ cho Web UI; muốn triển khai trên mạng thật cần bổ sung HTTPS, xác thực người dùng, kiểm soát truy cập và cơ chế kiểm toán hoạt động.

2.2 Thiết kế topology và luồng dữ liệu

2.2.1 Thành phần và quan hệ kết nối trong lab

Hệ thống mạng được thiết kế theo mô hình phân vùng (network segmentation) điển hình của doanh nghiệp vừa và nhỏ, với pfSense đóng vai trò gateway trung tâm kết nối và kiểm soát lưu lượng giữa các phân vùng.



Hình 2.1. Sơ đồ topology mạng tổng thể hệ thống NetMon SOC

2.2.2 Phân vùng mạng và quy hoạch địa chỉ

Hệ thống sử dụng không gian địa chỉ 10.20.0.0/16 cho toàn bộ mạng nội bộ lab, được phân chia thành các subnet chức năng như mô tả trong bảng sau:

Bảng 2.1. Quy hoạch địa chỉ IP theo phân vùng mạng

Phân vùng	Subnet	Gateway	Thiết bị / Mô tả
Management	10.20.0.0/24	-	Core-Router (10.20.0.1), SwL3A (10.20.0.2), SwL3B (10.20.0.3)
VLAN10 - IT	10.20.10.0/24	10.20.10.1	Người dùng phòng IT
VLAN20 - Sales	10.20.20.0/24	10.20.20.1	Người dùng phòng Kinh doanh
VLAN30 - HR	10.20.30.0/24	10.20.30.1	Người dùng phòng Nhân sự
VLAN40 - Engineering	10.20.40.0/24	10.20.40.1	Người dùng phòng Kỹ thuật

Phân vùng	Subnet	Gateway	Thiết bị / Mô tả
DMZ1 - Web	10.20.200.0/24	10.20.200.1	Server02 - Nginx Web Server (10.20.200.10)
DMZ2 - DNS/FTP	10.20.201.0/24	10.20.201.1	Server01 - BIND9 DNS + vsftpd FTP (10.20.201.10)
ZeroTier Overlay	172.24.0.0/16	-	pfSense ZeroTier (172.24.0.115), máy quản trị

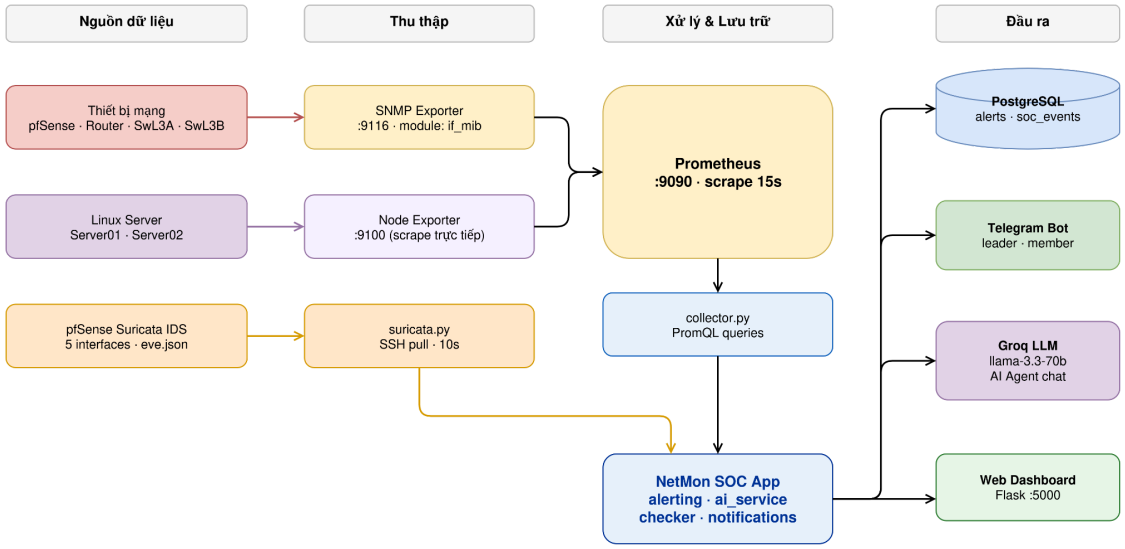
2.2.3 Luồng số liệu từ thiết bị tới ứng dụng

Các nguồn dữ liệu được chia thành ba luồng: metrics của router/switch/firewall qua SNMP Exporter, chỉ số Linux qua Node Exporter và sự kiện an ninh của Suricata qua EVE JSON. Hai nhóm metrics đi vào Prometheus, còn các alert IDS được chuẩn hóa và lưu trong PostgreSQL. Cách phân tách giúp truy vấn theo đặc trưng dữ liệu nhưng vẫn trình bày kết quả trên một giao diện.

Luồng 1 - SNMP Metrics: Prometheus SNMP Exporter (cổng 9116) định kỳ gửi yêu cầu SNMP GET đến 4 thiết bị mạng (pfSense, Core-Router, SwL3A, SwL3B) mỗi 15 giây. Kết quả được Prometheus lưu vào storage time-series. NetMon SOC query Prometheus HTTP API để lấy giá trị hiện tại và lịch sử.

Luồng 2 - Node Exporter Metrics: Prometheus scrape trực tiếp 2 Node Exporter chạy trên Server01 và Server02 (cổng 9100) mỗi 15 giây. Dữ liệu hệ thống Linux (CPU, RAM, Disk, Network) được lưu vào Prometheus.

Luồng 3 - Suricata IDS Events: Suricata worker trong NetMon SOC mở kết nối SSH đến pfSense (172.24.0.115) mỗi 10 giây, đọc N dòng cuối (N=500) từ các file eve.json của từng interface, parse JSON, lọc trùng lặp và lưu vào bảng soc_events của PostgreSQL.



Hình 2.2. Luồng dữ liệu giám sát trong hệ thống NetMon SOC

Luồng dữ liệu được kiểm chứng theo hướng từ nguồn đến nơi sử dụng: thiết bị xuất metric, Prometheus lưu mẫu, NetMon SOC truy vấn API và dashboard hiển thị; đồng thời Suricata ghi EVE JSON, worker lấy log qua kênh quản trị và lưu sự kiện đã chuẩn hóa. Hai luồng được đối chiếu bằng mốc thời gian và tên thiết bị, chứ không mặc nhiên được xem là một quan hệ nguyên nhân–kết quả.

2.3 Kiến trúc phần mềm và vòng đời cảnh báo

2.3.1 Cấu trúc các lớp chức năng

Hệ thống NetMon SOC được tổ chức theo kiến trúc phân lớp gồm 4 tầng rõ ràng:

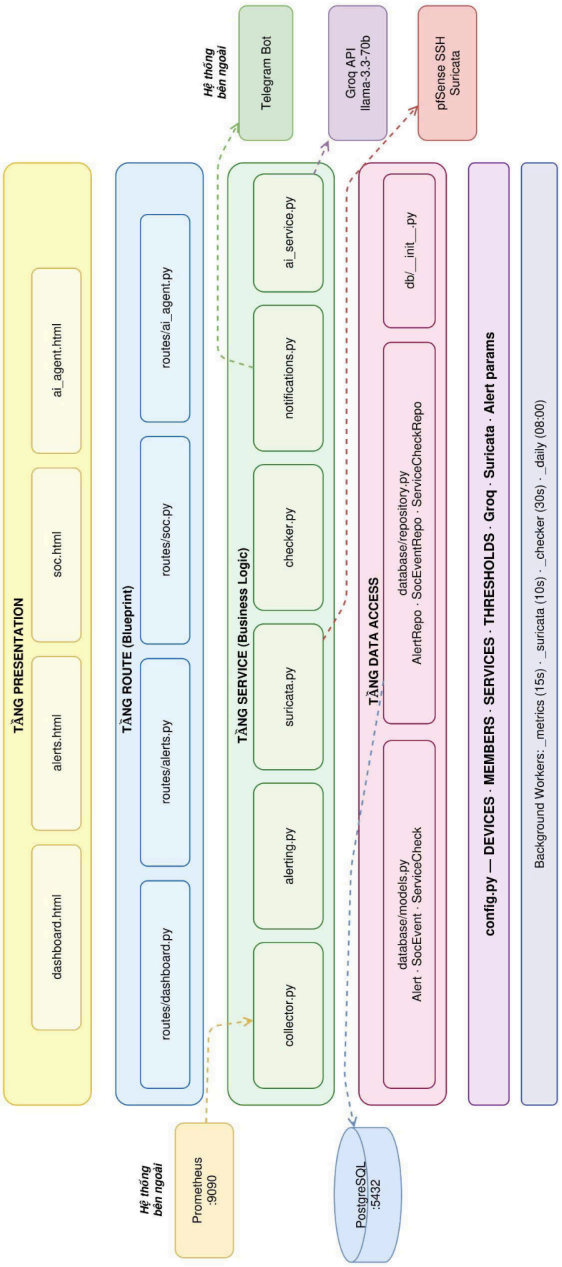
Tầng Presentation (Presentation Layer): Bao gồm các file Jinja2 template (HTML/CSS/JavaScript) trong thư mục templates/ và stylesheet trong static/. Tầng này chịu trách nhiệm hiển thị dữ liệu và tương tác người dùng. JavaScript frontend gọi các REST API endpoint để lấy dữ liệu và cập nhật giao diện mà không cần tải lại trang.

Tầng Route (Application Layer): Bao gồm các Blueprint trong thư mục routes/: dashboard.py, alerts.py, soc.py, ai_agent.py. Tầng này nhận HTTP request, gọi Service Layer để xử lý nghiệp vụ và trả về JSON response hoặc HTML template. Route không chứa logic nghiệp vụ hay truy vấn cơ sở dữ liệu trực tiếp.

Tầng Service (Business Logic Layer): Bao gồm các module trong thư mục services/: collector.py (thu thập metrics Prometheus), alerting.py (logic cảnh báo), suricata.py (tích

hợp IDS), checker.py (kiểm tra dịch vụ), notifications.py (gửi thông báo), ai_service.py (AI Agent và Isolation Forest), timezone.py (xử lý múi giờ). Đây là tầng quan trọng nhất chứa toàn bộ logic nghiệp vụ.

Tầng Data (Data Access Layer): Bao gồm database/models.py (định nghĩa ORM models), database/repository.py (tất cả query tập trung), database/__init__.py (khởi tạo engine và session factory). Tầng này cô lập hoàn toàn các thao tác cơ sở dữ liệu và áp dụng Repository Pattern.



Hình 2.3. Kiến trúc phân lớp của hệ thống NetMon SOC

2.3.2 Mô hình dữ liệu và tổ chức mã nguồn

Lược đồ cơ sở dữ liệu của NetMon SOC bao gồm 3 bảng chính, thiết kế tối giản nhưng đủ thông tin để phục vụ tất cả yêu cầu chức năng:

Bảng 2.2. Lược đồ bảng alerts - lưu trữ cảnh báo hệ thống

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	String(36)	PRIMARY KEY	UUID v4 định danh alert
created_at	DateTime	NOT NULL	Thời điểm tạo cảnh báo
device	String(64)	NOT NULL	Tên thiết bị (pfSense, Core-Router,...)
severity	String(16)	NOT NULL	Mức độ: critical, warning, info
category	String(32)	NOT NULL	Phân loại: cpu, memory, interface, suricata, ai-anomaly,...
message	Text	NOT NULL	Nội dung cảnh báo ngắn gọn
details	Text	-	Chi tiết bổ sung
sub_key	String(128)	-	Khóa phụ để debounce và tự động resolve
resolved	Boolean	NOT NULL	Trạng thái đã xử lý
resolved_by	String(64)	-	Ai đã xử lý (username hoặc "system")
resolved_at	DateTime	-	Thời điểm xử lý
resolved_note	Text	-	Ghi chú khi xử lý
resolve_mode	String(16)	-	Cách xử lý: human, auto, manual

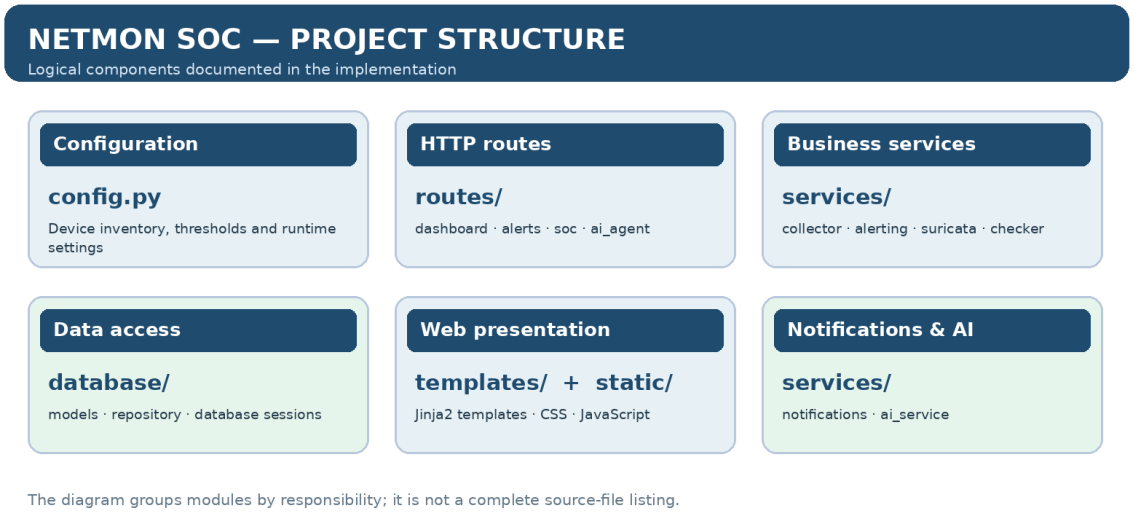
Bảng 2.3. Lược đồ bảng soc_events - lưu sự kiện Suricata IDS

Cột	Kiểu dữ liệu	Mô tả
id	BigInteger	Khóa chính tự tăng
occurred_at	DateTime	Thời điểm xảy ra sự kiện
src_ip / src_port	String(45) / Integer	Địa chỉ và cổng nguồn
dest_ip / dest_port	String(45) / Integer	Địa chỉ và cổng đích
proto	String(8)	Giao thức (TCP, UDP, ICMP)
severity	String(16)	Mức độ: critical, warning, info
priority	SmallInteger	Priority Suricata (1-4)

Cột	Kiểu dữ liệu	Mô tả
sig_id	Integer	Signature ID của luật kích hoạt
signature	Text	Tên chữ ký tấn công
category	String(64)	Danh mục chuẩn hóa
http_url / http_method / http_status	Text/String/SmallInt	Thông tin HTTP (nếu có)
interface_name	String(32)	Interface giám sát (WAN, Web-DMZ,...)
zone	String(32)	Vùng mạng (wan, dmz-web, lan,...)

Repository Pattern được áp dụng nhất quán trong toàn bộ tầng truy cập dữ liệu. Ba lớp repository được định nghĩa: AlertRepository, SocEventRepository, ServiceCheckRepository. Mỗi lớp chứa toàn bộ các phương thức truy vấn liên quan đến đối tượng đó, không có SQL nào nằm rải rác trong code nghiệp vụ.

Nguyên tắc tổ chức: mọi query đều đi qua context manager get_db() cung cấp session SQLAlchemy với tự động commit và rollback. Tất cả phương thức trả về kiểu Python thuần (dict, list) thay vì ORM object, tránh vấn đề lazy-loading và detached instance ngoài session.



Hình 2.4. Cấu trúc thư mục dự án

Các nhóm dữ liệu có đặc điểm khác nhau: Prometheus lưu chuỗi thời gian theo metric và nhãn; bảng `alerts` ghi nhận vòng đời một cảnh báo; bảng `soc_events` lưu sự kiện IDS gắn với thời điểm và chữ ký. Việc phân tách tránh sửa nội dung bằng chứng khi thay đổi trạng thái xử lý, đồng thời giúp quản lý thời hạn lưu trữ phù hợp với từng loại dữ liệu.

Các trường định danh như `device`, `sub_key`, `severity`, `status` và `created_at` hỗ trợ lọc sự cố theo nguồn và thời gian. Nếu nhiều worker cập nhật cùng một đối tượng, logic ghi dữ liệu phải tránh tạo hai cảnh báo trùng lặp. Lớp Repository tập trung truy vấn và giao dịch, giúp việc kiểm thử tính nhất quán không bị phân tán trong giao diện hoặc tác vụ nền.

2.3.3 Quy tắc tạo, nhắc lại và giải quyết cảnh báo

Vòng đời của một cảnh báo trong NetMon SOC trải qua các trạng thái: Counting → Confirmed → Active → Resolving → Resolved. Cơ chế được thiết kế để cân bằng giữa độ nhạy (sensitivity) và chống spam.

Giai đoạn Counting: Khi phát hiện điều kiện bất thường (ví dụ: CPU > 85%), hệ thống tăng bộ đếm lỗi (fail_count) cho sub_key tương ứng. Chỉ sau khi đạt PING_FAIL_THRESHOLD=3 lần lỗi liên tiếp, cảnh báo mới được xác nhận. Cơ chế này lọc bỏ các spike tức thời không đáng lo.

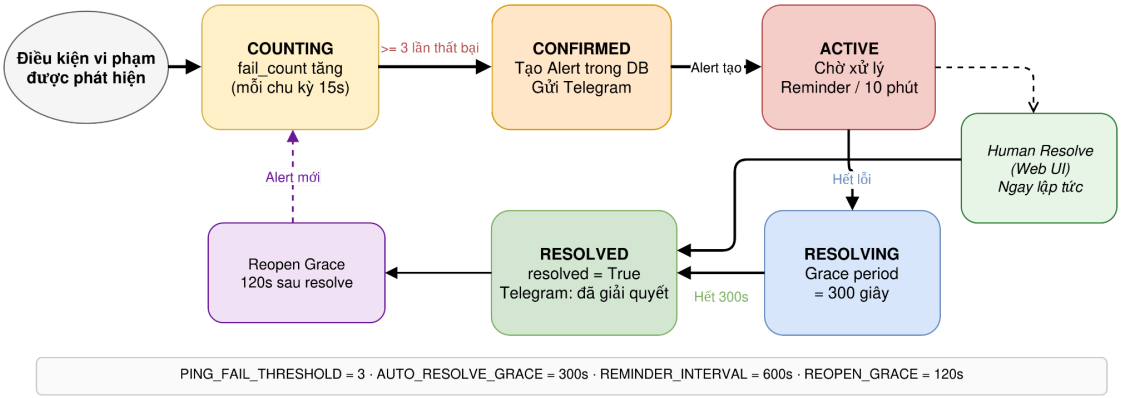
Giai đoạn Confirmed: Alert được tạo trong database, thông báo Telegram gửi đến thành viên phù hợp. Bộ đếm last_reminder được khởi tạo.

Giai đoạn Active: Nếu điều kiện bất thường vẫn tồn tại, hệ thống gửi thông báo nhắc lại mỗi REMINDER_INTERVAL_SECONDS=600 giây (10 phút) để đảm bảo không bị bỏ quên.

Giai đoạn Resolving: Khi điều kiện bất thường biến mất, fail_count reset về 0 và grace_deadline được đặt sau AUTO_RESOLVE_GRACE=300 giây (5 phút). Khoảng chờ này đảm bảo vấn đề thực sự đã được khắc phục, không phải chỉ tạm thời.

Giai đoạn Resolved: Sau khi grace period hết, alert được đánh dấu resolved với resolve_mode="auto". Thông báo "đã tự giải quyết" được gửi đến leader. Một

REOPEN_GRACE_SECONDS=120 giây được áp dụng trước khi alert mới cho cùng sub_key có thể được tạo, tránh vòng lặp cảnh báo-giải quyết-cảnh báo liên tục.



Hình 2.5. Vòng đời cảnh báo (Alert Lifecycle) trong hệ thống

Vòng đời của một cảnh báo không chỉ có hai trạng thái lỗi và bình thường. Trong giai đoạn bắt đầu, hệ thống tích lũy số lần vi phạm điều kiện; khi đạt debounce, một sự kiện được mở. Nếu lỗi còn kéo dài, hệ thống duy trì cùng nhận dạng alert và gửi nhắc lại theo chu kỳ thay vì phát sinh bản ghi mới không kiểm soát. Khi metric phục hồi, thời gian ổn định 300 giây được áp dụng trước khi tự giải quyết, sau đó khoảng chờ mở lại giúp hạn chế vòng lặp khi trạng thái chập chờn.

2.4 Thiết kế các cơ chế phát hiện và phân tích

2.4.1 Tương quan số liệu vận hành và sự kiện IDS

Dữ liệu Prometheus được lấy theo thiết bị, nhãn metric và thời điểm, trong khi Suricata trả sự kiện với địa chỉ nguồn, địa chỉ đích, chữ ký và mức nghiêm trọng. NetMon SOC đặt hai nguồn lên cùng giao diện nhằm hỗ trợ nhận biết rằng một CPU spike có xuất hiện gần thời điểm có nhiều kết nối bất thường hay không. Tuy nhiên, gần nhau về thời gian chỉ là dấu hiệu cần xem xét; hệ thống không khẳng định quan hệ nhân quả nếu thiếu bằng chứng bổ sung.

Đối với số liệu ngưỡng, hệ thống kết hợp điều kiện vượt ngưỡng và số lần vi phạm liên tiếp nhằm giảm thông báo do spike ngắn. Đối với Isolation Forest, điểm ngoại lai được dùng như tín hiệu bổ sung cho việc điều tra: sự lệch chuẩn thống kê chưa đồng nghĩa sự kiện đã được xác định là tấn công.

Bộ phát hiện ngưỡng sử dụng các mức CPU, RAM, Disk và trạng thái giao diện do người vận hành cấu hình. Isolation Forest bổ sung kết quả ngoại lai đối với bảy chỉ số được định nghĩa. Hai nhánh phát hiện có thể cùng tạo cảnh báo cho một sự kiện. Vì vậy, debounce, khóa nhận dạng sự kiện và giai đoạn chờ giải quyết có vai trò kiểm soát số lượng thông báo, nhưng không loại bỏ hoàn toàn false positive.

AI Agent nhận ngữ cảnh từ metrics và cảnh báo đã truy xuất, sau đó tạo phần giải thích hoặc gợi ý bước kiểm tra cho người vận hành. Đáp án của mô hình ngôn ngữ cần được đối chiếu với dữ liệu truy vấn và tình trạng thiết bị; hệ thống không cho phép mô hình tự kết luận hoặc áp dụng lệnh khắc phục mà không có người phê duyệt.

AI Agent không trực tiếp thu gói tin hay thay thế IDS. Khi nhận câu hỏi, ứng dụng tổng hợp trạng thái thiết bị từ Prometheus và một tập cảnh báo gần đây từ PostgreSQL, sau đó đưa bối cảnh vào yêu cầu gửi tới Groq API. Phản hồi có thể giải thích xu hướng, nêu giả thuyết và đề xuất lệnh kiểm tra; quyết định xử lý và việc xác minh tình trạng thực tế vẫn thuộc về người vận hành.

2.5 Định vị giải pháp và phạm vi so sánh

2.5.1 Các giải pháp tham khảo

Zabbix là giải pháp giám sát mạng mã nguồn mở được sử dụng rộng rãi trong doanh nghiệp, hỗ trợ SNMP, agent Zabbix, IPMI và nhiều phương thức thu thập khác. Ưu điểm của Zabbix là cộng đồng lớn, template phong phú, giao diện quản lý hoàn thiện và hỗ trợ tốt cho môi trường enterprise scale. Tuy nhiên, Zabbix không tích hợp sẵn tính năng phân tích bảo mật SOC, không có IDS integration, và không hỗ trợ phát hiện bất thường bằng học máy hay AI assistant trong phiên bản community.

Grafana kết hợp Prometheus là stack giám sát phổ biến trong cloud-native environments, cung cấp khả năng visualization mạnh mẽ và linh hoạt cao. Tuy nhiên, đây thuần túy là stack giám sát hiệu năng (performance monitoring), không có tính năng IDS, alerting tích hợp yêu cầu cấu hình phức tạp qua Grafana Alerting hoặc AlertManager, và không có AI Assistant.

Security Onion là bản phân phối Linux tập trung hoàn toàn vào an ninh mạng (NSM), tích hợp Zeek, Suricata, Elasticsearch và Kibana. Đây là giải pháp tập trung về khía cạnh

bảo mật nhưng yêu cầu phần cứng mạnh, cấu hình phức tạp và chủ yếu dành cho đội ngũ SOC chuyên biệt. Security Onion không phù hợp cho giám sát hiệu năng mạng thông thường và đòi hỏi chi phí vận hành cao.

2.5.2 Lựa chọn kiến trúc NetMon SOC

Bảng 2.4. So sánh NetMon SOC với các giải pháp hiện có

Tính năng	NetMon SOC	Zabbix	Grafana+Prometheus	Security Onion
SNMP Monitoring	Có	Có	Có	Không tích hợp sẵn*
Node Exporter	Có	Qua agent	Có	Không tích hợp sẵn*
IDS Integration	Suricata	Không tích hợp sẵn*	Không tích hợp sẵn*	Zeek+Suricata
Anomaly Detection ML	Isolation Forest	Không tích hợp sẵn*	Không tích hợp sẵn*	Không tích hợp sẵn*
AI Assistant (LLM)	Groq LLM	Không tích hợp sẵn*	Không tích hợp sẵn*	Không tích hợp sẵn*
Multi-channel Alert	Telegram	Email, Slack	AlertManager	Email
Alert Debounce	Có (configurable)	Có	Có	Không tích hợp sẵn*
Cấu hình đơn giản	1 file config.py	Qua web UI	Phức tạp	Rất phức tạp
Chi phí hạ tầng	Thấp	Trung bình	Thấp	Cao
Phù hợp SMB	Rất phù hợp	Phù hợp	Trung bình	Không phù hợp

Ghi chú: Dấu * phản ánh tích hợp mặc định trong phạm vi đối chiếu của dự án, không phải giới hạn tuyệt đối của hệ sinh thái sản phẩm. Các nền tảng có thể mở rộng bằng plugin hoặc công cụ bên ngoài.

CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG NETMON SOC

3.1 Chuẩn bị môi trường giả lập

3.1.1 Nền tảng, phiên bản và danh sách thiết bị

Toàn bộ hệ thống được triển khai trên một máy chủ vật lý (host machine) chạy PNETLab, đồng thời cũng là nơi chạy ứng dụng NetMon SOC, Prometheus và PostgreSQL. Cấu hình máy chủ đủ để xử lý tất cả workload: RAM đủ lớn cho các VM và ứng dụng, CPU nhiều nhân cho các background thread và máy ảo, ổ đĩa SSD đảm bảo I/O nhanh cho Prometheus TSDB và PostgreSQL.

Bảng 3.1. Phiên bản phần mềm sử dụng trong hệ thống

Thành phần	Phiên bản	Vai trò
PNETLab	Mới nhất	Nền tảng emulation mạng
pfSense	2.7.x (FreeBSD)	Firewall/Gateway/IDS host
Cisco IOSv	15.x	Router và Switch ảo
Ubuntu Server	22.04 LTS	Server01, Server02
Prometheus	2.x	TSDB và scraping
SNMP Exporter	0.x	Proxy SNMP → Prometheus
Node Exporter	1.x	Linux system metrics
Suricata	6.x / 7.x	IDS engine trên pfSense
PostgreSQL	15.x	Cơ sở dữ liệu
Python	3.12	Runtime NetMon SOC
Flask	2.3+	Web framework
scikit-learn	1.3+	Isolation Forest
ZeroTier	1.x	Overlay networking

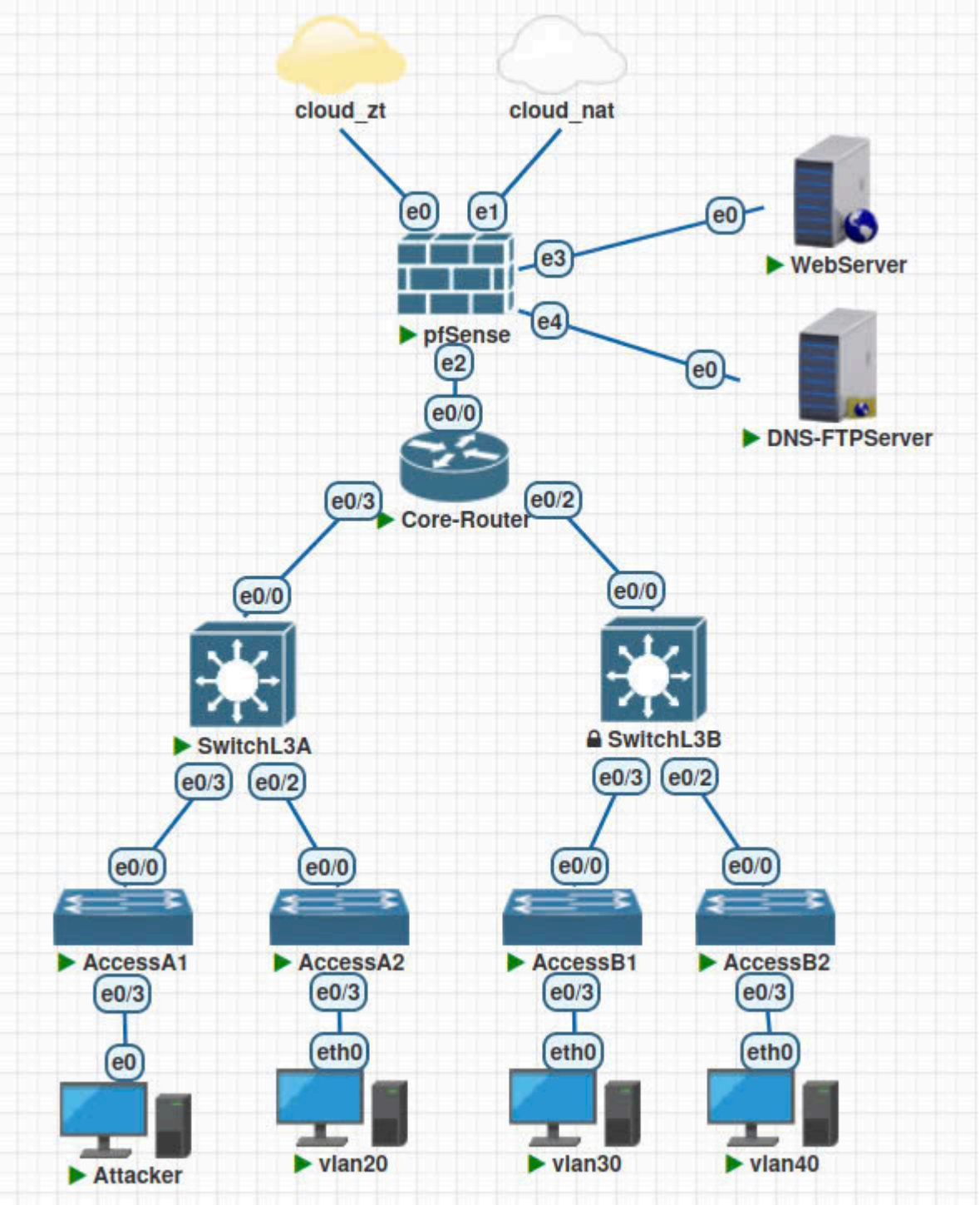
Bảng 3.2. Danh sách thiết bị trong hệ thống và vai trò chi tiết

Thiết bị / Loại	Địa chỉ IP	Vai trò	Phụ trách
pfSense — Firewall	172.24.0.115 (ZT)	Firewall, NAT, VPN ZeroTier, Suricata IDS	tuan

Thiết bị / Loại	Địa chỉ IP	Vai trò	Phụ trách
Core-Router — Router (Cisco IOSv)	10.20.0.1	OSPF backbone, liên kết các phân vùng	vuong
SwL3A — Switch L3 (Cisco IOSv)	10.20.0.2	VLAN10 IT, VLAN20 Sales, DHCP	vuong
SwL3B — Switch L3 (Cisco IOSv)	10.20.0.3	VLAN30 HR, VLAN40 Engineering, DHCP	tuan
Server02 — Linux (Ubuntu)	10.20.200.10	Web Server Nginx, DMZ1	vuong
Server01 — Linux (Ubuntu)	10.20.201.10	DNS BIND9 + FTP vsftpd, DMZ2	vuong

3.1.2 Topology thực tế trên PNETLab

Topology trong PNETLab gồm 6 node ảo kết nối qua các virtual bridge Linux nội bộ của PNETLab, phản ánh chính xác cấu trúc mạng doanh nghiệp với phân vùng bảo mật rõ ràng.



Hình 3.1. Topology mạng trong PNETLab

3.1.3 pfSense, các vùng mạng và ZeroTier

Cấu hình interface: pfSense có ít nhất 5 interface vật lý ảo được cấu hình: WAN (kết nối ra ngoài / Internet), LAN (kết nối nội bộ đến Core-Router), Web-DMZ, DNS-FTP và ZeroTier. Mỗi interface được gán địa chỉ IP tĩnh tương ứng với quy hoạch mạng.

Cấu hình NAT: Quy tắc NAT Masquerade được thiết lập cho tất cả lưu lượng từ mạng nội bộ ra WAN, cho phép các thiết bị trong lab truy cập Internet qua địa chỉ IP của pfSense.

Cấu hình Firewall Rules: Quy tắc tường lửa được thiết lập theo nguyên tắc least privilege: mạng VLAN người dùng chỉ được phép truy cập DMZ Web (cổng 80, 443) và DMZ DNS (cổng 53); các mạng VLAN không được kết nối trực tiếp với nhau; lưu lượng từ ngoài vào DMZ bị kiểm soát chặt.

Cấu hình SNMP: SNMP v2c được kích hoạt trên pfSense với community string (cấu hình trong biến môi trường để bảo mật). Prometheus SNMP Exporter scrape pfSense qua địa chỉ ZeroTier 172.24.0.115 với module if_mib.

Cấu hình Suricata: Suricata được cài đặt qua pfSense Package Manager và cấu hình ở chế độ IDS (không chặn, chỉ ghi log) trên tất cả 5 interface. Ruleset ET Open (Emerging Threats Open) được sử dụng làm bộ luật cơ sở. EVE JSON log được bật với full detail cho alert, dns, http, tls events.

Cấu hình ZeroTier: ZeroTier client được cài đặt trên pfSense và join vào network ID của nhóm. Địa chỉ ZeroTier 172.24.0.115 được gán cố định cho pfSense trong ZeroTier Control Panel. Firewall rule cho phép máy quản trị (ZeroTier peer) SSH vào pfSense để thu thập log Suricata.

3.2 Thiết lập mạng lõi và dịch vụ thử nghiệm

Các node hạ tầng được triển khai theo vai trò: router lõi định tuyến giữa các vùng; switch L3 phục vụ VLAN người dùng; máy chủ Web, DNS và FTP tạo các endpoint phục vụ giám sát và kiểm thử an ninh. Những dịch vụ này đồng thời cung cấp điểm đích để kiểm tra trạng thái ứng dụng thay vì chỉ kiểm tra ICMP.

3.2.1 Router lõi và định tuyến OSPF

Core-Router chạy Cisco IOSv với giao thức định tuyến OSPF (Open Shortest Path First) để phân phối thông tin routing đến các thiết bị trong mạng. Các interface của Core-Router bao gồm liên kết đến pfSense, liên kết đến SwL3A, SwL3B và các interface routed đến DMZ1, DMZ2.

Cấu hình OSPF sử dụng area 0 (backbone area) cho tất cả các liên kết, đơn giản hóa cấu hình trong môi trường lab. SNMP community string được cấu hình trên router để Prometheus SNMP Exporter có thể thu thập metrics.

```
! Cấu hình SNMP cơ bản trên Cisco IOSv
snmp-server community <community-string> RO
snmp-server ifindex persist


! Cấu hình OSPF
router ospf 1
 network 10.20.0.0 0.0.255.255 area 0
 passive-interface default
 no passive-interface GigabitEthernet0/0  ! Interface kết nối pfSense
```

3.2.2 VLAN và dịch vụ DHCP trên hai switch L3

SwL3A và SwL3B là các switch Layer 3 Cisco IOSv chịu trách nhiệm phân phối kết nối đến các VLAN người dùng và cung cấp DHCP service.

SwL3A quản lý VLAN10 (10.20.10.0/24 - IT) và VLAN20 (10.20.20.0/24 - Sales), mỗi VLAN có SVI (Switched Virtual Interface) với gateway tương ứng và DHCP pool riêng. SwL3B quản lý VLAN30 (10.20.30.0/24 - HR) và VLAN40 (10.20.40.0/24 - Engineering) theo cấu hình tương tự.

Cấu hình SNMP được thiết lập giống Core-Router, cho phép Prometheus giám sát trạng thái interface và lưu lượng qua giao thức SNMP v2c.

3.2.3 Linux Server cung cấp Web, DNS và FTP

Server02 (10.20.200.10 - Web Server): Ubuntu Server 22.04 LTS với Nginx cài đặt làm web server phục vụ nội dung HTTP cơ bản trong vùng DMZ1. Node Exporter chạy tại cổng 9100 để Prometheus thu thập system metrics.

Server01 (10.20.201.10 - DNS + FTP): Ubuntu Server 22.04 LTS với BIND9 làm DNS server phân giải tên miền nội bộ lab.local (ví dụ: web.lab.local → 10.20.200.10) và vsftpd làm FTP server trong vùng DMZ2. Node Exporter cũng chạy tại cổng 9100.

```
# Tải Node Exporter

wget
https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_e
xporter-1.7.0.linux-amd64.tar.gz

tar xvf node_exporter-*.tar.gz



# Cài đặt service systemd

sudo cp node_exporter /usr/local/bin/

sudo useradd -rs /bin/false node_exporter



# /etc/systemd/system/node_exporter.service

sudo systemctl enable --now node_exporter
```

3.3 Triển khai pipeline giám sát và IDS

3.3.1 Prometheus và SNMP Exporter

Prometheus và SNMP Exporter được cài đặt trên máy chủ host (nơi chạy PNETLab). File cấu hình prometheus.yml định nghĩa 6 scrape job như đã mô tả trong thiết kế, với scrape_interval: 15s và evaluation_interval: 15s.

```
global:
  scrape_interval:     15s
  evaluation_interval: 15s
scrape_configs:
  - job_name: "snmp-pfsense"
    metrics_path: /snmp
    params:
      module: [if_mib]
      target: ["172.24.0.115"]
    static_configs:
      - targets: ["localhost:9116"]
        labels: {device: "pfSense", role: "firewall"}
  - job_name: "node-server02"
    static_configs:
```

SNMP Exporter chạy tại cổng 9116, đóng vai trò proxy chuyển đổi SNMP response sang định dạng Prometheus metrics. Module if_mib được sử dụng để thu thập toàn bộ metrics từ IF-MIB.

3.3.2 Node Exporter và kiểm tra endpoint metrics

Node Exporter được cài đặt trên Server01 và Server02 theo quy trình chuẩn. Sau khi khởi động, endpoint /metrics tại cổng 9100 xuất hàng trăm metrics hệ thống. Prometheus xác nhận scrape thành công khi truy cập http://localhost:9090/targets và thấy trạng thái UP cho cả hai job node-server01 và node-server02.

3.3.3 Suricata IDS và thu thập EVE JSON

Suricata được cài đặt qua menu System → Package Manager → Available Packages trên pfSense WebGUI. Sau khi cài đặt, cấu hình được thực hiện trong menu Services → Suricata:

Thêm Interface: Mỗi interface (WAN, Web-DMZ, DNS-FTP, LAN, ZeroTier) được thêm vào danh sách giám sát với cấu hình riêng. Chế độ hoạt động được đặt là Legacy Mode (không inline, không chặn gói tin).

Cấu hình EVE JSON logging: Trong tab Logging Settings của mỗi interface, bật EVE JSON log với đầy đủ loại sự kiện (alert, dns, http, tls, flow). Đường dẫn lưu file eve.json theo định dạng /var/log/suricata/suricata_<interface_id>/eve.json.

Cài đặt Ruleset: ET Open ruleset được cập nhật qua internet. Các category rule được kích hoạt bao gồm: emerging-attack_response, emerging-exploit, emerging-malware, emerging-scan, emerging-web_client, emerging-web_server.

Xác minh hoạt động: Sau khi khởi động, kiểm tra Suricata đang chạy bằng lệnh trên pfSense console:

```
ps aux | grep suricata

tail -f /var/log/suricata/suricata_vtnet125952/eve.json
```

Cấu hình SSH key-based auth: Tạo SSH key pair trên máy chủ host, copy public key vào pfSense qua System → User Manager → Users → admin → Authorized SSH Keys. Xác nhận kết nối SSH hoạt động không cần mật khẩu từ host.

Sau khi tạo từng interface Suricata, việc xác minh không chỉ là nhìn tiến trình ở trạng thái running. Cần kiểm tra file EVE JSON có bản ghi mới, trường `event_type` và `timestamp` hợp lệ, worker đọc từ vị trí tiếp nối đúng và database không ghi trùng một sự kiện khi worker khởi động lại. Một đường truyền quản trị bị gián đoạn có thể khiến thời điểm phát sinh sự kiện và thời điểm NetMon SOC nhập sự kiện chênh nhau, cần dùng timestamp gốc để trình bày chính xác trên timeline.

3.4 Xây dựng backend, mô hình và cảnh báo

Mã ứng dụng NetMon SOC được tổ chức theo các module chức năng. Sau khi cài dependencies và khai báo tham số môi trường, hệ thống khởi tạo database, đăng ký worker lấy dữ liệu và phục vụ giao diện web. Những khóa API, token thông báo và mật khẩu kết nối chỉ được khai báo qua biến môi trường, không đưa vào báo cáo.

3.4.1 Khởi tạo ứng dụng Flask và cơ sở dữ liệu

```
cd /opt/netmon

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt
```

Cấu hình môi trường: Tạo file.env từ template.env.example và điền các thông tin cần thiết:

```
DATABASE_URL=postgresql://netmon:netmon_pass@localhost:5432/netmon

PROMETHEUS_URL=http://localhost:9090

TELEGRAM_BOT_TOKEN=<bot_token>

TELEGRAM_VUONG_ID=<chat_id_vuong>

TELEGRAM_TUAN_ID=<chat_id_tuan>

GROQ_API_KEY=<groq_api_key>

SURICATA_SSH_HOST=172.24.0.115

SURICATA_SSH_USER=admin

SURICATA_SSH_KEY=/home/minhvuong/.ssh/id_rsa
```

```
# Tạo PostgreSQL database và user

sudo -u postgres psql -c "CREATE USER netmon WITH PASSWORD 'netmon_pass';"

sudo -u postgres psql -c "CREATE DATABASE netmon OWNER netmon;"


# Chạy script setup tự động

bash scripts/setup_db.sh
```

```
python app.py

# Output:

# [env] Đã load 12 biến từ /opt/netmon/.env

# [netmon-soc] http://0.0.0.0:5000

# [worker:metrics] started

# [worker:suricata] started

# [worker:checker] started
```

Sau khi khởi động, ứng dụng có thể truy cập qua trình duyệt tại http://<host-ip>:5000. Giao diện landing page hiển thị danh sách thiết bị và thành viên nhóm.

3.4.2 Huấn luyện và nạp mô hình Isolation Forest

Sau khi hệ thống chạy ổn định ít nhất 24 giờ (tốt nhất là 7 ngày) để tích lũy dữ liệu lịch sử trong Prometheus, tiến hành huấn luyện mô hình Isolation Forest:

```
cd /opt/netmon

python scripts/train_models.py
```

Script tự động xác định loại thiết bị và chọn metric phù hợp để train: traffic (in/out bps) và error rate cho các thiết bị mạng; CPU%, RAM%, Disk% cho server. Dữ liệu được lấy trong khoảng 7 ngày (168 giờ) với bước 5 phút từ Prometheus, qua đó thu thập khoảng 2016 điểm mẫu mỗi metric.

Sau khi huấn luyện thành công, 7 file mô hình được tạo ra trong thư mục models/:

```
models/
├── isolation_forest_cpu_percent.pkl          (628 KB)
├── isolation_forest_mem_used_pct.pkl        (1.28 MB)
├── isolation_forest_disk_used_pct.pkl       (1.40 MB)
├── isolation_forest_traffic_in_bps.pkl      (1.32 MB)
├── isolation_forest_traffic_out_bps.pkl     (1.25 MB)
├── isolation_forest_in_errors.pkl           (216 KB)
└── isolation_forest_out_errors.pkl          (119 KB)
```

Kích thước file phản ánh lượng dữ liệu training và độ phức tạp mô hình (200 cây). Mỗi file chứa cả đối tượng IsolationForest và StandardScaler được lưu cùng nhau bằng joblib.dump.

Khởi động lại ứng dụng sau khi có mô hình để AI Service tải và kích hoạt anomaly detection:

```
python app.py

# [ai] Loaded model: cpu_percent

# [ai] Loaded model: mem_used_pct

# ...
```

Quy trình sử dụng một khoảng lịch sử Prometheus làm baseline. Trước khi huấn luyện cần kiểm tra các trường hợp không có dữ liệu, tập mẫu quá ngắn hoặc biến động do tác động kiểm thử vô tình lọt vào baseline; những điểm này có thể khiến ngưỡng ngoại lai phản ánh trạng thái thử nghiệm thay vì hoạt động bình thường. Sau khi tạo các

tệp mô hình, ứng dụng nạp mô hình và bộ chuẩn hóa đúng cặp tương ứng với từng metric. Thử nghiệm với spike lớn có tác dụng xác nhận pipeline hoạt động, nhưng chưa đánh giá độ nhạy với bất thường nhỏ hoặc chậm.

3.4.3 Tích hợp thông báo và xử lý trạng thái alert

Ứng dụng đọc giá trị metric, so sánh với ngưỡng và trạng thái lịch sử để quyết định tạo hoặc cập nhật alert. Khi điều kiện được xác nhận qua số chu kỳ liên tiếp, thông báo được gửi theo vai trò thiết bị phụ trách. Với những lỗi đã phục hồi, bộ đếm ổn định được duy trì trước khi hệ thống tự đánh dấu resolved. Các giá trị thử nghiệm được sử dụng gồm debounce ba chu kỳ, thời gian chờ giải quyết 300 giây và nhắc lại sau 600 giây.

Tuyến thông báo cần giữ được thông tin tối thiểu: loại sự kiện, thiết bị/endpoint, giá trị hoặc dấu hiệu dẫn đến cảnh báo, thời gian xảy ra và trạng thái xử lý. Mỗi thông báo nên có một khóa sự kiện để kỹ thuật viên biết đó là cảnh báo mới, nhắc lại hay thông báo đã phục hồi. Trong triển khai lab, việc phân định leader/member nhằm tránh gửi tất cả thông báo cho mọi người; tuy nhiên, phân quyền người nhận tin nhắn không tương đương phân quyền truy cập ứng dụng web.

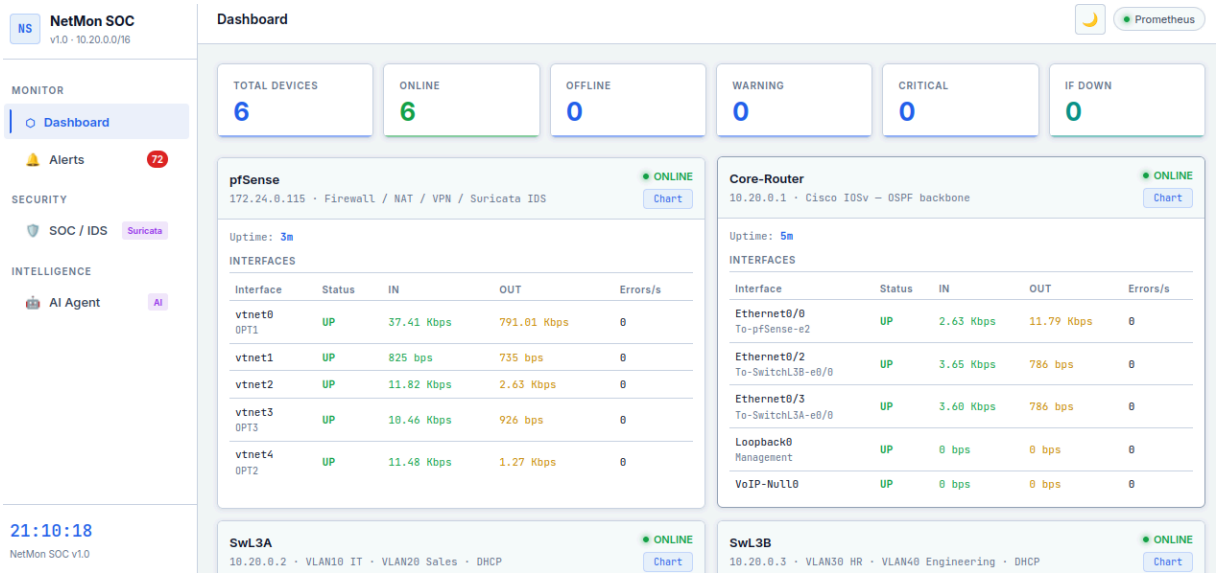
3.5 Giao diện và hành trình của người vận hành

3.5.1 Dashboard thiết bị và trang quản lý cảnh báo

Dashboard là giao diện chính của hệ thống, hiển thị tổng quan trạng thái toàn bộ hạ tầng mạng trong thời gian thực. Dữ liệu được tự động làm mới mỗi REFRESH_INTERVAL=15 giây thông qua AJAX polling đến endpoint /api/devices.

Phần tóm tắt trạng thái phía trên dashboard hiển thị 4 chỉ số tổng hợp: tổng số thiết bị, số thiết bị đang hoạt động (online), số interface down và số dịch vụ có vấn đề. Màu sắc theo code: xanh lá (bình thường), vàng (cảnh báo), đỏ (nghiêm trọng).

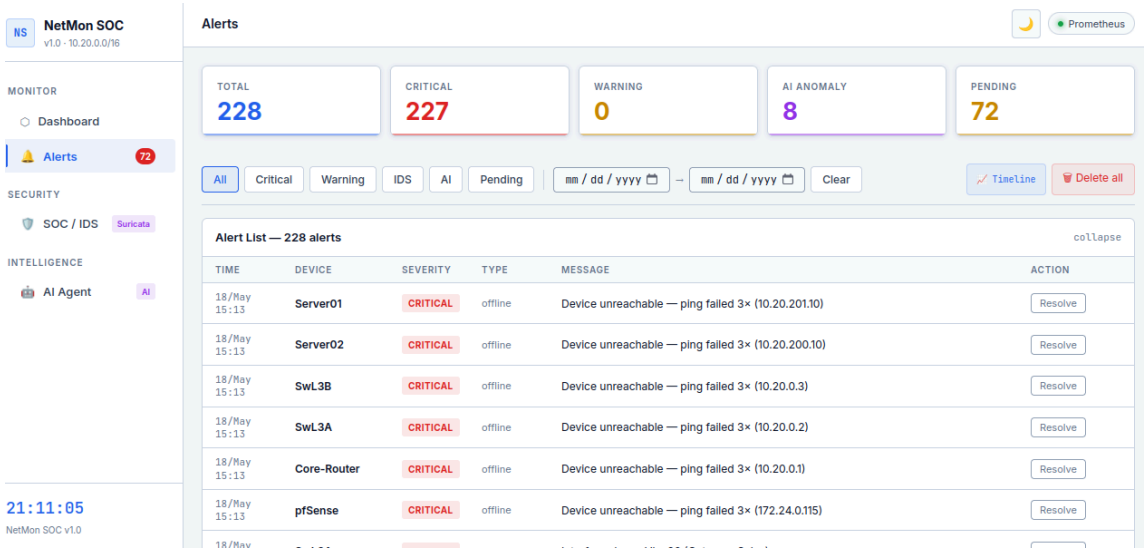
Mỗi thiết bị được hiển thị trong một card riêng, bao gồm: tên và loại thiết bị, địa chỉ IP, trạng thái online/offline, thời gian uptime, danh sách interface với trạng thái và lưu lượng, biểu đồ lưu lượng theo thời gian. Khi nhấn vào một thiết bị, panel mở rộng hiển thị đồ thị chi tiết lưu lượng 60 phút gần nhất.



Hình 3.2. Giao diện Dashboard giám sát thiết bị thời gian thực

Trang Alerts cung cấp giao diện quản lý toàn bộ cảnh báo trong hệ thống với khả năng lọc đa chiều. Các bộ lọc bao gồm: theo mức độ nghiêm trọng (critical, warning, info), theo trạng thái (đang mở / đã giải quyết), theo thiết bị và theo khoảng thời gian.

Phần thống kê đầu trang hiển thị: tổng số alert, số critical, số warning, số alert từ AI anomaly, số đang chưa xử lý. Bảng alert chi tiết hiển thị timestamp, thiết bị, mức độ, phân loại, nội dung và trạng thái. Mỗi alert có nút "Resolve" cho phép người dùng đánh dấu đã xử lý với ghi chú tùy chọn. Biểu đồ timeline hiển thị phân bố cảnh báo theo thời gian để nhận diện xu hướng.



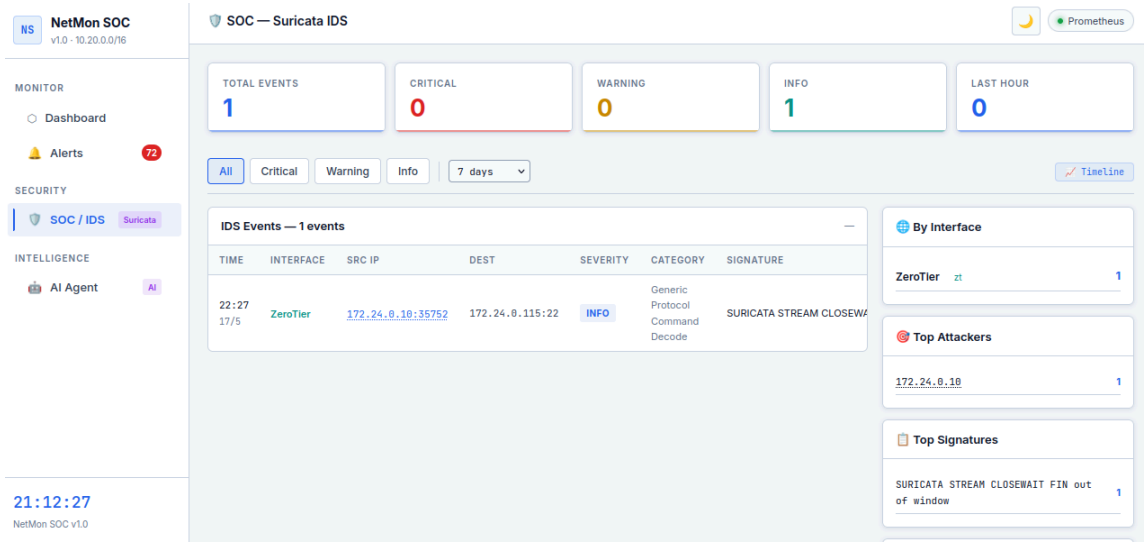
Hình 3.3. Giao diện trang quản lý cảnh báo (Alerts Management)

3.5.2 SOC Dashboard và AI Agent

Trang SOC là giao diện phân tích bảo mật tập trung, hiển thị toàn bộ sự kiện IDS được thu thập từ Suricata trên tất cả 5 interface. Trang được thiết kế theo phong cách Security Operations Center chuyên nghiệp.

Phần tổng quan (Overview) hiển thị các chỉ số trong 24 giờ: tổng sự kiện, số critical, số warning, top IP tấn công, phân bố theo phân vùng mạng (zone distribution). Biểu đồ cột timeline phân theo giờ giúp xác định thời điểm xảy ra nhiều sự kiện nhất.

Bảng sự kiện chi tiết hỗ trợ lọc theo: mức độ nghiêm trọng, phân loại tấn công (Port Scan, Web Attack, DoS,...), địa chỉ IP nguồn, interface và vùng mạng. Mỗi dòng sự kiện hiển thị đầy đủ thông tin: thời gian, IP nguồn:port, IP đích:port, giao thức, chữ ký tấn công và phân loại. Nhấn vào một IP nguồn sẽ hiển thị panel phân tích chi tiết về attacker đó: tổng sự kiện, phân bố danh mục tấn công, danh sách IP đích đã tấn công, lịch sử hoạt động.



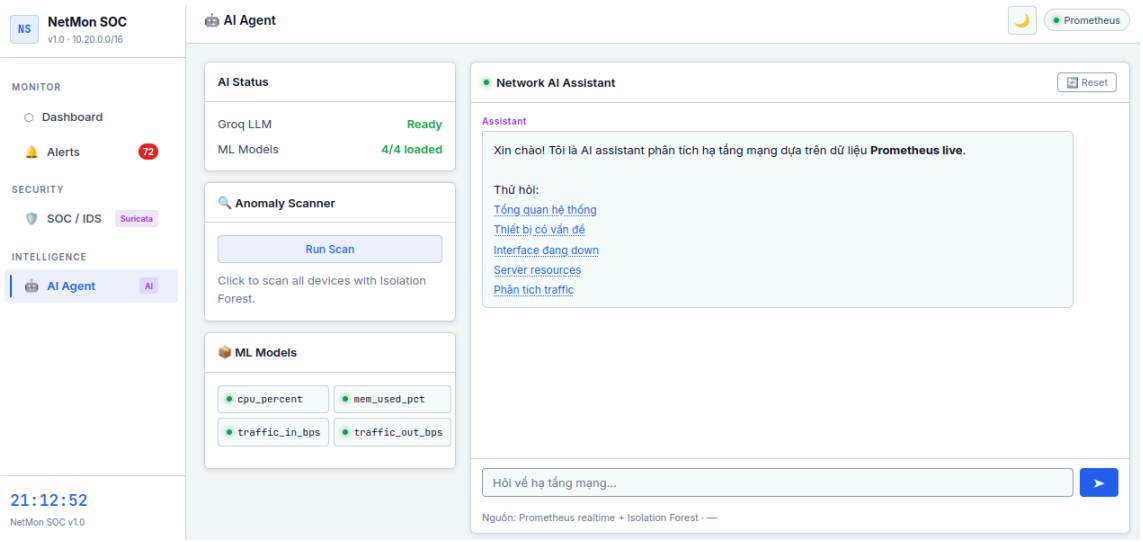
Hình 3.4. Giao diện SOC Dashboard phân tích sự kiện Suricata IDS

Trang AI Agent cung cấp giao diện hội thoại (chat interface) với AI assistant được tích hợp Groq LLM. Giao diện tương tự chat messenger với lịch sử hội thoại cuộn, thanh nhập liệu và nút gửi.

Mỗi khi người dùng gửi câu hỏi, hệ thống tự động thu thập context bao gồm: trạng thái hiện tại của tất cả thiết bị từ Prometheus (metrics gần thời gian thực), danh sách 10 cảnh báo gần nhất từ database, tóm tắt tổng quan hạ tầng. Tất cả context này được đưa

vào system prompt trước khi gọi Groq API, đảm bảo AI trả lời dựa trên tình trạng thực tế của hệ thống.

AI có thể trả lời các loại câu hỏi: "CPU của Server02 đang ở mức nào?", "Thiết bị nào đang có cảnh báo?", "Giải thích lỗi interface down trên SwL3A", "Lệnh gì để kiểm tra OSPF neighbor trên Core-Router?". AI cũng được tích hợp với anomaly detection: khi phát hiện bất thường, AI tự động gợi ý nguyên nhân và hướng xử lý.



Hình 3.5. Giao diện AI Agent với phân tích ngữ cảnh mạng thời gian thực

Đối với thông báo, cần kiểm tra cả bản ghi lưu trữ và tin Telegram nhận được, sau đó tạo một tình huống phục hồi để xác nhận trạng thái resolved. Với AI Agent, kết quả có giá trị khi các metric được nêu trong phản hồi có thể đối chiếu lại với Prometheus tại thời điểm tương ứng. Đây là nguyên tắc kiểm tra tính nhất quán giữa quan sát đầu vào và thông tin mà hệ thống cung cấp cho người vận hành.

CHƯƠNG 4. KIỂM THỬ, KẾT QUẢ VÀ PHÂN TÍCH

4.1 Phương pháp và phạm vi kiểm thử

4.1.1 Môi trường, công cụ và quy tắc đánh giá

Kiểm thử được thực hiện trong môi trường mô phỏng PNETLab đã triển khai, sử dụng máy Kali Linux (kết nối vào mạng lab qua ZeroTier) làm máy tấn công. Các công cụ kiểm thử bao gồm:

stress-ng: Tạo tải CPU/RAM nhân tạo trên server

iperf3: Kiểm thử băng thông mạng

4.1.2 Bộ kịch bản kiểm thử

Bảng 4.1. Tổng hợp các kịch bản kiểm thử

Mã	Kịch bản / Công cụ	Thiết bị mục tiêu	Kết quả mong đợi
TC01	Thiết bị offline Công cụ: Tắt nguồn VM	pfSense, Router, Switch	Alert "Device offline" sau 3 lần ping fail
TC02	Interface down Công cụ: Tắt interface	SwL3A GE0/1	Alert "Interface down"
TC03	CPU cao Công cụ: stress-ng	Server02	Alert "CPU > 85%" + AI anomaly
TC04	RAM cao Công cụ: stress-ng	Server01	Alert "RAM > 90%" + AI anomaly
TC05	Disk đầy Công cụ: Tạo file lớn	Server02	Alert "Disk > 85%"
TC06	Port Scan Công cụ: Nmap	Server01, Server02	Suricata alert "Port Scan"
TC07	SQL Injection Công cụ: SQLMap	Server02 (Nginx)	Suricata alert "Web Attack"
TC08	Directory Traversal Công cụ: curl	Server02	Suricata alert "Web Attack"
TC09	Brute Force SSH Công cụ: Hydra	Server01	Suricata alert "Brute Force"

Mã	Kịch bản / Công cụ	Thiết bị mục tiêu	Kết quả mong đợi
TC10	DoS nhẹ Công cụ: hping3	pfSense WAN	Suricata alert "DoS"
TC11	Traffic bất thường Công cụ: iperf3 flood	Router/Switch	AI anomaly alert
TC12	Telegram notification Công cụ: TC01	Toàn hệ thống	Nhận Telegram message
TC13	Auto-resolve Công cụ: Khắc phục TC01	pfSense	Alert tự giải quyết sau 5 phút

Bộ 13 kịch bản chính kiểm tra các tuyến chức năng quan trọng của NetMon SOC: giám sát trạng thái, thu thập tài nguyên, phát hiện dấu hiệu IDS, học máy và vòng đời cảnh báo. Các menu sinh lưu lượng WAN, Web, FTP và DNS là biến thể thử nghiệm mở rộng; chúng không được cộng thêm vào số lượng 13 kịch bản nghiệm thu. Mỗi bài thử được đối chiếu trên ba điểm: thao tác tạo sự kiện, bằng chứng hệ thống nhận được và kết quả hiển thị hoặc thông báo.

4.2 Kiểm thử giám sát tình trạng hạ tầng

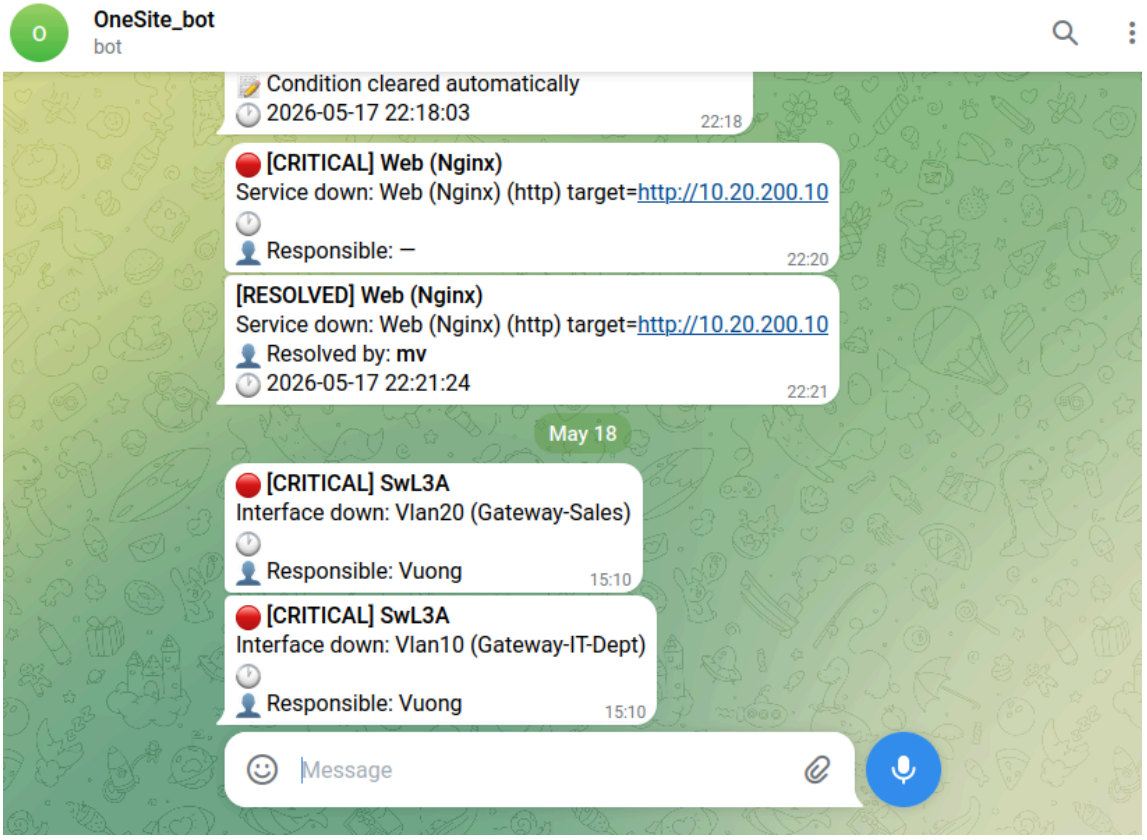
Hai nhóm lỗi được tách riêng: ngắt kết nối thiết bị/giao diện và tăng tải tài nguyên khi thiết bị vẫn hoạt động. Độ trễ được xem xét theo chuỗi tác động → lần lấy mẫu hoặc đọc log → điều kiện tạo alert → thông báo; không được đồng nhất tổng thời gian này với chu kỳ Prometheus scrape.

4.2.1 Thiết bị offline và giao diện bị ngắt

Thực hiện: Trong PNETLab, tắt nguồn VM pfSense. Quan sát hệ thống trong 3 chu kỳ ping (45 giây).

Kết quả: Sau 45 giây (3 lần ping thất bại liên tiếp), alert "pfSense không phản hồi ICMP ping" được tạo trong database với severity="critical". Thông báo Telegram được gửi đến cả leader (Vuong) và member phụ trách (Tuan). Trên Dashboard, card pfSense chuyển sang màu đỏ với trạng thái "offline".

Khi khởi động lại pfSense, hệ thống phát hiện thiết bị online sau chu kỳ tiếp theo (15 giây). Grace period 300 giây bắt đầu đếm. Sau 5 phút không có lỗi tiếp theo, alert được auto-resolve và thông báo "resolved" gửi qua Telegram.



Hình 4.1. Thông báo Telegram khi pfSense offline và tự động giải quyết

Thực hiện: Trên SwL3A, shutdown interface GigabitEthernet0/1 (interface được đặt alias "VLAN10-Uplink"):

```
SwL3A(config)# interface GigabitEthernet0/1

SwL3A(config-if)# shutdown
```

Kết quả: Trong vòng 30 giây (2 chu kỳ scrape tiếp theo), SNMP Exporter thu thập ifOperStatus=2 (down) cho interface này. Logic trong alerting.py phát hiện interface có alias, trạng thái admin UP nhưng oper DOWN - điều kiện tạo cảnh báo. Alert "Interface GE0/1 (VLAN10-Uplink) trên SwL3A ở trạng thái DOWN" được tạo với severity="warning". Thông báo Telegram gửi đến member "vuong" (phụ trách SwL3A).

Các phép thử này sử dụng hai cơ chế phát hiện khác nhau. Trạng thái offline được suy ra từ một chuỗi ping thất bại, còn interface down được phát hiện bằng `ifOperStatus`

so với `ifAdminStatus` qua SNMP. Vì thế, thời gian ghi nhận khác nhau là hệ quả của điều kiện kích hoạt, lịch lấy mẫu và logic xử lý tương ứng; không nên lấy một kết quả riêng làm cam kết cho tất cả loại sự cố.

4.2.2 Tài nguyên CPU, RAM và dung lượng đĩa

Kiểm thử CPU: Chạy stress-ng trên Server02 để đẩy CPU lên >85%:

stress-ng --cpu 4 --timeout 120s

Kết quả CPU: Trong vòng 30-45 giây sau khi CPU vượt ngưỡng, hệ thống tạo alert "CPU Server02 = 92.3% (ngưỡng 85%)". Đồng thời, module Isolation Forest phát hiện anomaly: score = -0.68 (< -0.55, ngưỡng critical), tạo thêm alert "AI Anomaly: cpu_percent bất thường" với severity="critical". Như vậy một sự kiện CPU cao tạo ra hai loại alert bổ sung cho nhau: threshold-based và ML-based.

Ở bài thử RAM, nhóm tạo tải bằng `stress-ng --vm 1 --vm-bytes 80%`. Mức sử dụng khoảng 80% thấp hơn ngưỡng RAM > 90% nên không đủ điều kiện phát cảnh báo theo ngưỡng. Nếu điểm dữ liệu vẫn lệch đáng kể so với baseline, Isolation Forest có thể đưa ra cảnh báo ngoại lai; hai loại cảnh báo cần được đối chiếu riêng theo kết quả quan sát trong bảng.

Bảng 4.2. Kết quả kiểm thử ngưỡng tài nguyên server

Kịch bản	Giá trị đo / Ngưỡng	Thời gian phát hiện	Loại cảnh báo
CPU stress Server02	Đo: 92.3% Ngưỡng: 85%	~30 giây	Threshold + AI Anomaly
RAM stress Server01	Đo: 80.2% Ngưỡng: 90%	N/A	AI Anomaly only
Disk đầy Server02	Đo: 87.5% Ngưỡng: 85%	~15 giây	Threshold
CPU bình thường	Đo: 12.4% Ngưỡng: -	-	Không alert

4.3 Kiểm thử phát hiện xâm nhập theo vùng mạng

Thử nghiệm IDS được tổ chức theo phân vùng mạng để làm rõ lưu lượng phát sinh ở đâu, Suricata quan sát trên interface nào và sự kiện được hiển thị thế nào. Các mẫu

Nmap, SQLMap, Hydra, hping3 và các công cụ tạo yêu cầu thử nghiệm chỉ được sử dụng trong topology có kiểm soát; kết quả có ý nghĩa xác minh đường xử lý log và luật phát hiện đã bật, không phải phép kiểm định năng lực phát hiện mọi kiểu tấn công.

4.3.1 Biên WAN, quét cổng và lưu lượng từ chối dịch vụ mức thấp

Trong nhóm này, nhóm tạo các mẫu dò quét cổng và lưu lượng tăng đột ngột tới vùng WAN để kiểm tra Suricata trên pfSense. Bài thử được đánh giá qua chữ ký ghi nhận, địa chỉ nguồn–đích, thời điểm và sự xuất hiện của bản ghi tương ứng trong SOC Dashboard. Tình trạng có một alert không đồng nghĩa tấn công đã thành công hoặc được ngăn chặn.

Các menu trong NetMon Attack Console tạo tình huống theo từng bề mặt: WAN (quét và lưu lượng mạnh), Web (request mang chuỗi thử nghiệm bất thường), FTP (xác thực hoặc lệnh dịch vụ) và DNS (tần suất truy vấn, nhãn dài, loại bản ghi bất thường). Cách phân nhóm này giúp người đọc đối chiếu tín hiệu với vị trí quan sát, đồng thời tránh coi tên một công cụ là đủ để xác định bản chất của sự kiện.

Thực hiện từ máy Kali Linux trong mạng lab VLAN10, nhóm tiến hành quét toàn diện vào Server01 nhằm kiểm tra khả năng phát hiện hành vi reconnaissance của Suricata IDS.

```
nmap -sV -sC -p- --min-rate=1000 10.20.201.10
```

Kết quả cho thấy Suricata trên interface LAN ghi nhận nhiều cảnh báo thuộc nhóm Port Scan hoặc Detection of a Network Scan. Trong vòng khoảng 10–20 giây, worker của hệ thống SOC pull events từ Suricata và lưu vào bảng soc_events. Trên giao diện SOC Dashboard, các sự kiện từ IP nguồn của máy Kali được gom thành cụm, phân loại đúng là hành vi quét cổng với mức độ cảnh báo tương ứng.

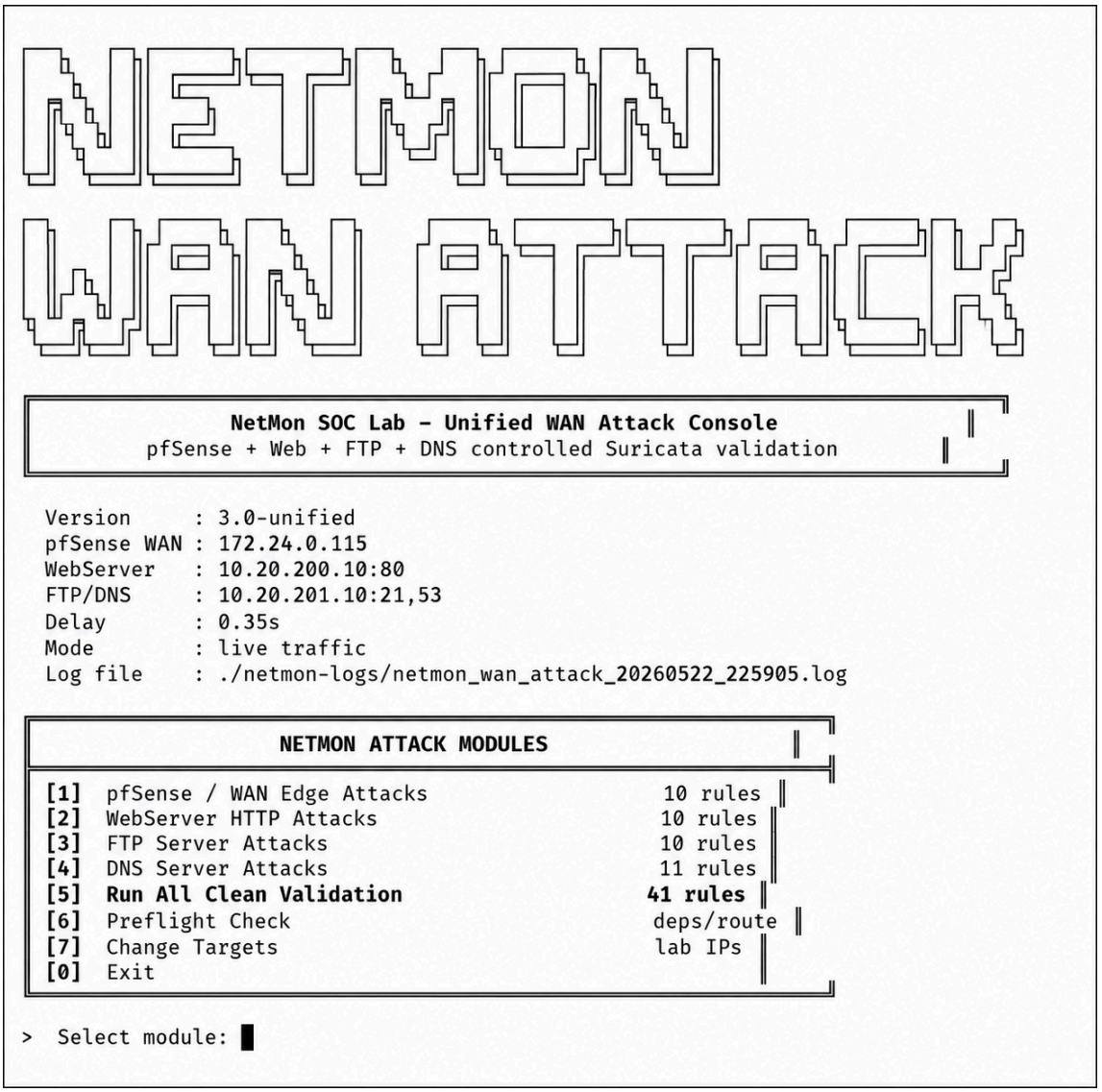
Thực hiện: Gửi lưu lượng SYN flood cường độ thấp vào pfSense WAN:

```
hping3 -S --flood -V -p 443 172.24.0.115
```

Kết quả: Suricata interface WAN (zone: wan) phát hiện "Attempted Denial of Service" với chữ ký liên quan đến SYN flood pattern. Do cường độ thấp (môi trường

lab, không có traffic thật), impact thực tế lên pfSense không đáng kể nhưng IDS vẫn ghi nhận đúng loại tấn công.

Giao diện chính của NetMon cho phép lựa chọn nhiều nhóm kiểm thử, bao gồm tấn công vào vùng WAN/pfSense, tấn công Web Server, tấn công FTP Server và tấn công DNS Server. Mỗi nhóm lại bao gồm nhiều kịch bản nhỏ tương ứng với các hành vi thường gặp trong thực tế. Nhờ vậy, hệ thống SOC không chỉ được kiểm tra với một vài tình huống đơn lẻ mà còn được đánh giá trên nhiều bề mặt tấn công khác nhau.



Hình 4.2. Giao diện chính NetMon WAN Attack Console

Ở giao diện chính, công cụ hiển thị các thông tin quan trọng như phiên bản, địa chỉ WAN pfSense, địa chỉ WebServer, địa chỉ FTP/DNS Server, chế độ chạy live traffic và file log ghi nhận quá trình kiểm thử. Các module được chia thành nhiều nhóm như WAN Edge Attacks, WebServer HTTP Attacks, FTP Server Attacks, DNS Server Attacks và

Run All Clean Validation. Việc chia module rõ ràng giúp quá trình kiểm thử dễ theo dõi, đồng thời thuận tiện khi cần chạy riêng từng nhóm tấn công để đối chiếu với log Suricata.

Đối với vùng WAN, NetMon Attack Console mô phỏng các hành vi nhằm kiểm tra khả năng phát hiện tấn công tại biên mạng pfSense. Đây là vị trí quan trọng vì mọi lưu lượng từ bên ngoài đi vào hệ thống đều phải đi qua firewall trước khi tiếp cận các vùng mạng nội bộ hoặc DMZ.

Các kịch bản WAN có thể bao gồm dò quét dịch vụ, kiểm tra phản hồi cổng, tạo lưu lượng bất thường, mô phỏng SYN burst hoặc các hành vi giống DoS mức độ thấp. Khi chạy nhóm kiểm thử này, Suricata trên interface WAN sẽ ghi nhận các alert tương ứng, sau đó hệ thống SOC tiến hành thu thập, chuẩn hóa và hiển thị trên Dashboard.

```
hping3 -S -p 443 --flood 172.24.0.115
nmap -sS -Pn -p 1-1000 172.24.0.115
```

4.3.2 Các dấu hiệu tấn công ứng dụng Web

SQL Injection với SQLMap và Directory Traversal thủ công:

```
sqlmap -u "http://10.20.200.10/?id=1" --dbs --batch
curl "http://10.20.200.10/../../../../etc/passwd"
curl "http://10.20.200.10/admin/../../../etc/shadow"
```

Kết quả: Suricata interface Web-DMZ (zone: dmz-web) ghi nhận các alert category "Web Application Attack" với chữ ký cụ thể như "ET WEB_SERVER SQL Injection Attempt" và "ET WEB_SERVER Possible Directory Traversal". Severity được phân loại "warning" đến "critical" tùy theo priority của luật. Thông tin HTTP URL và method (GET/POST) được ghi vào bảng soc_events, hỗ trợ forensics phân tích chi tiết URL đã được probe.

Đối với Web Server, nhóm sử dụng giao diện Web Attack Generator để mô phỏng nhiều dạng tấn công ứng dụng Web khác nhau. Các kịch bản bao gồm brute force login,

SQL Injection, XSS, Command Injection, Path Traversal/LFI, Webshell Access, File Upload Attack, Directory Enumeration và Light DoS/SYN Burst Test.



Hình 4.3. Giao diện Web Attack Generator

```
curl "http://10.20.200.10/?id=1' OR '1'='1"
curl "http://10.20.200.10/search?q=<script>alert(1)</script>"
curl "http://10.20.200.10/../../../../etc/passwd"
curl "http://10.20.200.10/admin/"
```

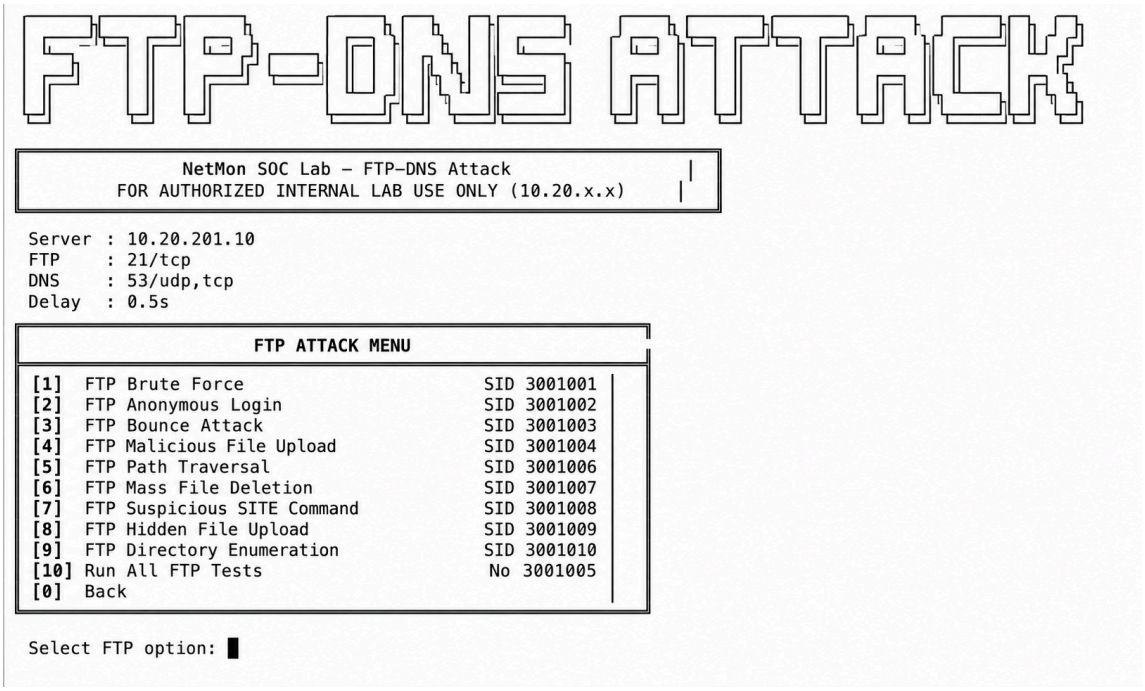
Kết quả kiểm thử cho thấy Suricata có thể phát hiện nhiều nhóm cảnh báo thuộc loại Web Application Attack. Các cảnh báo này được lưu lại với thông tin chi tiết như URL, method, IP nguồn, IP đích và chữ ký phát hiện. Đây là dữ liệu quan trọng để SOC Dashboard hiển thị timeline, thống kê attacker và hỗ trợ phân tích forensics.

4.3.3 Dịch vụ FTP, DNS và bảng tổng hợp IDS

```
hydra -l root -P /usr/share/wordlists/rockyou.txt -t 4 ssh://10.20.201.10
```

Kết quả: Suricata phát hiện pattern brute force SSH qua chữ ký "ET SCAN Potential SSH Scan" và "SURICATA STREAM 3way handshake anomaly". Số lượng lớn attempt trong thời gian ngắn tạo ra cụm sự kiện dễ nhận diện trên timeline SOC. Alert Telegram được gửi với severity="critical" vì priority Suricata = 1 cho brute force alerts.

Đối với FTP Server, công cụ NetMon cung cấp nhiều kịch bản kiểm thử như FTP Brute Force, FTP Anonymous Login, FTP Bounce Attack, Malicious File Upload, Path Traversal, Mass File Deletion, Suspicious SITE Command, Hidden File Upload và Directory Enumeration.



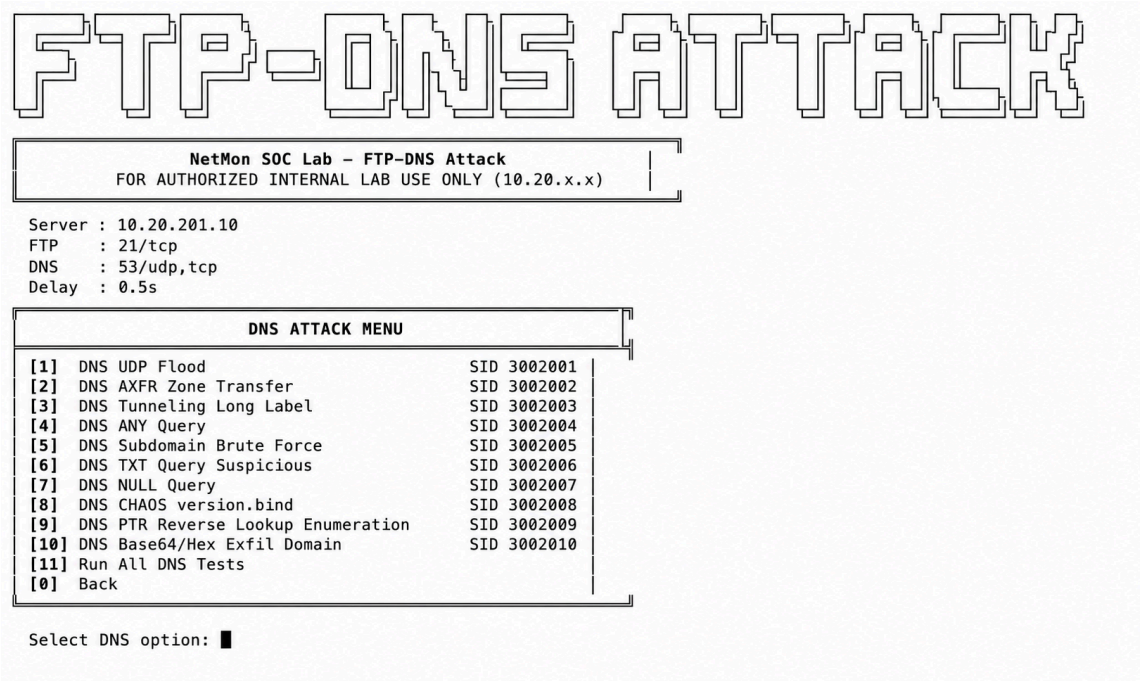
Hình 4.4. Giao diện FTP Attack Menu

Nhóm kiểm thử FTP giúp đánh giá khả năng phát hiện các hành vi tấn công vào dịch vụ truyền file. Đây là dịch vụ thường xuất hiện trong hệ thống nội bộ và có nguy cơ bị khai thác nếu cấu hình yếu, cho phép anonymous login hoặc bị lộ tài khoản. Việc bổ sung nhóm kịch bản FTP giúp báo cáo thể hiện được phạm vi kiểm thử rộng hơn, không chỉ tập trung vào Web và SSH.

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://10.20.201.10
ftp 10.20.201.10
curl ftp://anonymous:anonymous@10.20.201.10/
curl -T shell.php ftp://user:password@10.20.201.10/uploads/
```

Kết quả mong đợi là Suricata ghi nhận các cảnh báo liên quan đến brute force, đăng nhập bất thường, upload file đáng ngờ hoặc hành vi liệt kê thư mục. Các sự kiện này được gửi về hệ thống SOC và phân loại vào nhóm FTP Attack hoặc Brute Force tùy theo chữ ký được kích hoạt.

Đối với DNS Server, NetMon bổ sung nhiều kịch bản chuyên biệt như DNS UDP Flood, AXFR Zone Transfer, DNS Tunneling Long Label, DNS ANY Query, Subdomain Brute Force, Suspicious TXT Query, NULL Query, CHAOS version.bind, PTR Reverse Lookup Enumeration và Base64/Hex Exfil Domain.



Hình 4.5. Giao diện DNS Attack Menu

Nhóm DNS mở rộng phạm vi kiểm thử sang lưu lượng hạ tầng ứng dụng, bên cạnh các giao thức Web và FTP. Các tình huống với truy vấn TXT, nhãn dài hoặc mẫu truy vấn lặp được tạo nhằm kiểm tra khả năng lưu và phân loại sự kiện của Suricata. Dữ liệu ghi nhận giúp xác định nguồn yêu cầu và mẫu truy vấn cần xem xét, không tự chứng minh truyền dữ liệu trái phép.

```
dig @10.20.201.10 axfr lab.local
dig @10.20.201.10 ANY lab.local
dig @10.20.201.10 TXT suspicious.lab.local
dig @10.20.201.10 $(echo "secretdata" | base64).lab.local
```

Trong các tình huống DNS đã mô phỏng, dashboard ghi nhận sự kiện tương ứng với các mẫu truy vấn bất thường được tạo. Việc phân loại chính xác cần xem chữ ký kích hoạt, bối cảnh dịch vụ hợp lệ và mẫu quan sát đủ dài; bản báo cáo chỉ kết luận hệ thống có thể ghi nhận và hiển thị những tình huống đã thử.

Bảng 4.3. Tổng hợp kết quả kiểm thử Suricata IDS sau khi mở rộng

STT	Kịch bản / Công cụ	Vùng và hành vi mô phỏng	Kết quả ghi nhận
1	Port Scan Công cụ: Nmap	LAN Quét cổng, dò dịch vụ	Suricata phát hiện nhóm Port Scan / Network Scan
2	SQL Injection Công cụ: SQLMap, curl, NetMon Web Attack	Web-DMZ Chèn truy vấn SQL bất thường	Ghi nhận Web Application Attack
3	Directory Traversal / LFI Công cụ: curl, NetMon Web Attack	Web-DMZ Truy cập đường dẫn nhạy cảm	Ghi nhận cảnh báo Path Traversal / LFI
4	XSS Simulation Công cụ: NetMon Web Attack	Web-DMZ Payload script trong request	Ghi nhận cảnh báo Web Attack
5	Command Injection Công cụ: NetMon Web Attack	Web-DMZ Payload thực thi lệnh	Ghi nhận cảnh báo Web Application Attack
6	Webshell / File Upload Công cụ: NetMon Web Attack	Web-DMZ Upload hoặc truy cập file đáng ngờ	Ghi nhận sự kiện HTTP bất thường
7	Brute Force SSH Công cụ: Hydra	LAN / Server Zone Thử mật khẩu liên tục	Ghi nhận cụm sự kiện brute force / scan

STT	Kịch bản / Công cụ	Vùng và hành vi mô phỏng	Kết quả ghi nhận
8	FTP Brute Force Công cụ: Hydra, NetMon FTP Attack	FTP-DNS Zone Thử đăng nhập FTP	Ghi nhận cảnh báo brute force FTP
9	FTP Anonymous Login Công cụ: ftp, curl, NetMon FTP Attack	FTP-DNS Zone Đăng nhập anonymous	Ghi nhận truy cập FTP bất thường
10	FTP Upload / Directory Enumeration Công cụ: curl, ftp, NetMon FTP Attack	FTP-DNS Zone Upload file, liệt kê thư mục	Ghi nhận FTP Attack / suspicious activity
11	DNS AXFR Zone Transfer Công cụ: dig, NetMon DNS Attack	FTP-DNS Zone Thử chuyển vùng DNS	Ghi nhận DNS Attack / suspicious query
12	DNS Tunneling / Long Label Công cụ: dig, NetMon DNS Attack	FTP-DNS Zone Truy vấn domain bất thường	Ghi nhận DNS tunneling pattern
13	DNS ANY / TXT / NULL Query Công cụ: dig, NetMon DNS Attack	FTP-DNS Zone Truy vấn DNS bất thường	Ghi nhận DNS suspicious query
14	DNS Exfiltration Simulation Công cụ: dig, NetMon DNS Attack	FTP-DNS Zone Mô phỏng rò rỉ dữ liệu qua domain	Ghi nhận domain bất thường
15	DoS / SYN Flood mức độ thấp Công cụ: hping3, NetMon WAN/Web Attack	WAN / Web-DMZ SYN burst, DoS nhẹ	Ghi nhận Attempted Denial of Service

Một lần đăng nhập FTP thất bại không đủ xác định brute force; tương tự, truy vấn DNS mang nhãn dài hoặc TXT có thể xuất hiện trong ứng dụng hợp lệ. Vì vậy, kết quả IDS được hiểu như dấu hiệu để người vận hành điều tra, không thay thế đối chiếu lưu lượng, log máy chủ và bằng chứng thực thi trước khi kết luận.

4.4 Kiểm thử học máy và vòng đời cảnh báo

Nhóm thử nghiệm học máy làm rõ sự khác nhau giữa cảnh báo ngưỡng và phát hiện ngoại lai từ dữ liệu lịch sử. Tải CPU, RAM hoặc lưu lượng được tạo có kiểm soát, sau đó kiểm tra điểm bất thường, bản ghi cảnh báo và màn hình giám sát. Điểm ML chỉ xác nhận độ lệch so với baseline của metric, không phân loại chắc chắn nguyên nhân.

4.4.1 Bất thường lưu lượng mạng

Thực hiện: Sử dụng iperf3 để tạo luồng traffic lớn bất thường giữa hai server:

```
# Trên Server01 (iperf server)

iperf3 -s

# Từ máy Kali (iperf client)

iperf3 -c 10.20.201.10 -b 500M -t 60
```

Kết quả: Lưu lượng 500 Mbps vượt xa mức baseline bình thường của router trong lab (thường < 10 Mbps). Sau khi metric traffic_in_bps được thu thập bởi Prometheus và truy vấn bởi module anomaly detection, Isolation Forest cho điểm số: score = -0.71 (< -0.55 = threshold critical). Alert "AI Anomaly: traffic_in_bps bất thường (score=-0.71)" được tạo với severity="critical".

4.4.2 Biến động CPU/RAM và cảnh báo đa lớp

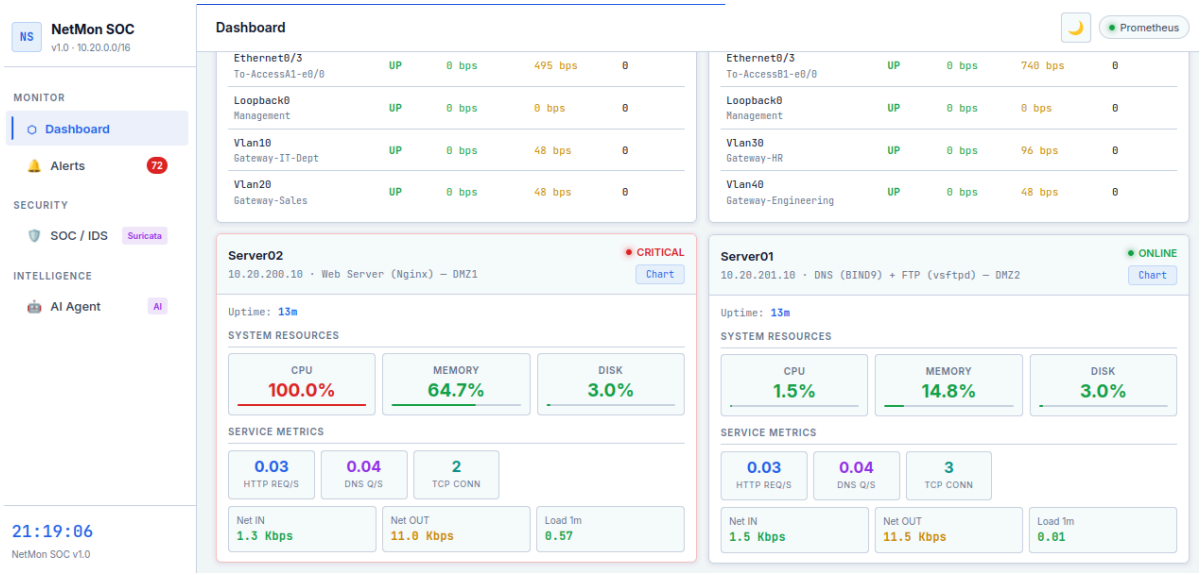
```
# Tải CPU đột biến trên Server02

stress-ng --cpu 4 --timeout 180s &

# Đồng thời tải RAM

stress-ng --vm 2 --vm-bytes 75% --timeout 180s &
```

Trong bài thử tải đồng thời trên Server02, CPU tăng từ khoảng 8% lên 96,4%, RAM từ khoảng 35% lên 87,2%. Mô hình ghi nhận điểm ngoại lai tương ứng cho CPU (-0,69) và RAM (-0,42); với ngưỡng cấu hình của lab, hệ thống phát cảnh báo AI theo mức độ khác nhau. Đây là tác động tải tổng hợp, không phải bằng chứng máy chủ đã nhiễm mã độc; cảnh báo theo ngưỡng CPU được kích hoạt riêng vì vượt 85%.



Hình 4.6. Alert AI Anomaly phát hiện bất thường CPU và RAM trên Server02

4.4.3 Telegram, debounce, reminder và auto-resolve

Kịch bản: Kích hoạt cảnh báo CPU cao trên Server02. Quan sát thông báo Telegram nhận được trên hai thiết bị: điện thoại của "vuong" (role: leader) và "tuan" (role: member).

Kết quả: Vuong (leader) nhận được thông báo đầy đủ với nội dung: loại cảnh báo, thiết bị, giá trị thực đo, thời điểm và hướng xử lý gợi ý. Tuan (member) nhận được thông báo tóm tắt ngắn hơn vì Server02 không phải thiết bị trong phụ trách của Tuan nhưng vẫn nhận vì category "critical". Tin nhắn được định dạng HTML hợp lệ với các ký tự đặc biệt được escape đúng cách.

Kiểm thử nhắc nhở (reminder): Sau khi alert được tạo và không được xử lý, sau 10 phút (600 giây) hệ thống tự động gửi tin nhắc lại qua Telegram với nội dung "[REMINDER] Cảnh báo chưa được xử lý:...". Nhắc nhở tiếp tục mỗi 10 phút cho đến khi alert được resolve.

Kiểm thử debounce: Kích hoạt tải CPU 90% nhưng chỉ trong 20 giây (ít hơn 3 chu kỳ = 45 giây). Kết quả: không có alert nào được tạo vì fail_count chưa đạt ngưỡng 3. Cơ chế debounce hoạt động đúng, lọc bỏ spike tức thời.

Kiểm thử auto-resolve: Kích hoạt CPU cao liên tục để alert được tạo (sau 3 chu kỳ). Sau đó giảm tải về mức bình thường. Quan sát: sau khi fail_count reset về 0, grace_deadline được đặt 300 giây sau. Sau đúng 5 phút, alert tự chuyển sang resolved

với resolve_mode="auto" và resolve_by="system". Thông báo auto-resolve gửi qua Telegram đến leader.

Bảng 4.4. Kết quả kiểm thử vòng đời alert

Kịch bản	Hành vi mong đợi	Kết quả thực tế	Đạt/Không
Spike CPU 20s	Không tạo alert (debounce)	Không tạo alert	Đạt
CPU cao liên tục > 45s	Tạo alert sau 3 fail	Alert tạo sau ~45s	Đạt
Nhắc lại sau 10 phút	Telegram reminder	Nhận message reminder	Đạt
CPU trở về bình thường	Auto-resolve sau 5 phút	Resolved sau ~5 phút	Đạt
Human resolve	Alert marked resolved ngay	Resolved ngay lập tức	Đạt
Reopen grace 2 phút	Alert mới sau >= 2 phút	Đúng	Đạt

4.5 Đánh giá kết quả và giới hạn số liệu

Bảng tổng hợp dưới đây đối chiếu các kịch bản chính với trạng thái ghi nhận; phần phân tích tiếp theo tập trung vào mức độ tin cậy của bằng chứng. Các con số độ trễ và phản hồi API thuộc điều kiện đo trong lab; không nên dùng chúng như cam kết vận hành hoặc như chỉ số độ chính xác phát hiện nếu chưa có tập đánh giá độc lập.

4.5.1 Kết quả các kịch bản chính

Bảng 4.5. Tổng hợp kết quả toàn bộ kịch bản kiểm thử

Mã TC	Kịch bản	Kết quả	Ghi chú
TC01	Thiết bị offline	Đạt	Phát hiện sau 3 chu kỳ ping (~45s)
TC02	Interface down	Đạt	Phát hiện qua SNMP ifOperStatus
TC03	CPU > 85%	Đạt	Threshold alert + AI anomaly
TC04	RAM > 80%	Đạt	AI anomaly (không vượt ngưỡng cứng)
TC05	Disk > 85%	Đạt	Threshold alert
TC06	Port Scan	Đạt	Suricata phát hiện đúng category

Mã TC	Kịch bản	Kết quả	Ghi chú
TC07	SQL Injection	Đạt	Suricata phát hiện đúng
TC08	Directory Traversal	Đạt	Suricata phát hiện đúng
TC09	Brute Force SSH	Đạt	Alert critical từ Suricata
TC10	DoS SYN Flood	Đạt	Phát hiện trên interface WAN
TC11	Traffic bất thường	Đạt	AI anomaly score -0.71
TC12	Telegram notification	Đạt	Phân quyền nhận đúng vai trò
TC13	Auto-resolve	Đạt	Resolve sau grace period 300s

4.5.2 Độ trễ phát hiện và hiệu năng API

Thời gian phản hồi API: Được đo bằng cách gọi các endpoint chính lặp lại 100 lần và tính trung bình:

/api/devices (lấy toàn bộ device snapshots): trung bình 85ms (có cache), 650ms (không cache khi cache TTL hết)

/api/soc/events (100 sự kiện gần nhất): trung bình 45ms

/api/soc/stats (thống kê 24 giờ): trung bình 120ms

Tài nguyên hệ thống: NetMon SOC ở trạng thái hoạt động bình thường tiêu thụ khoảng 150-200MB RAM và dưới 5% CPU trên máy host. PostgreSQL với khoảng 10,000 sự kiện soc_events tiêu thụ thêm khoảng 100MB RAM cho buffer pool. Tổng tài nguyên nhẹ, không ảnh hưởng đến các VM trong PNETLab.

Bảng 4.6. Độ trễ phát hiện của từng loại cảnh báo

Loại cảnh báo	Chu kỳ poll	Thời gian phát hiện thực tế	Ghi chú
Device offline	15s (ping)	45-60 giây	3 fail * 15s + xử lý
Interface down	15s (SNMP)	15-30 giây	1-2 chu kỳ scrape
CPU/RAM threshold	15s	15-30 giây	Sau khi vượt ngưỡng
Suricata alert	10s (SSH pull)	10-20 giây	1-2 chu kỳ pull
AI anomaly	15s	15-30 giây	Cùng chu kỳ metrics

Độ trễ phải được diễn giải theo chu kỳ lấy mẫu, thời điểm tạo bản ghi và thời điểm cập nhật giao diện hoặc Telegram. Các giá trị trong bảng là quan sát của các trường hợp thử nghiệm, chưa phải phân bố thống kê như P95/P99 hoặc kết quả đo dưới tải liên tục. Trung bình 100 yêu cầu API chỉ minh họa khả năng đáp ứng tại cấu hình này.

Cache với TTL 12 giây có thể giúp truy vấn lại dữ liệu nhanh hơn so với lúc cần lấy mẫu mới từ Prometheus. Vì vậy, so sánh thời gian phản hồi giữa trạng thái cache hit và cache miss phải nêu rõ điều kiện; các con số hiện có không chứng minh năng lực phục vụ số lượng lớn người dùng đồng thời.

4.5.3 Cảnh báo sai và độ tin cậy của kết luận

Một đánh giá định lượng đáng tin cậy cần định nghĩa rõ "sự kiện thật", "báo động giả", số mẫu được gắn nhãn và quy tắc chia tập dữ liệu. Các kết quả hiện có chứng minh đường xử lý sự kiện của lab vận hành trong một số tình huống cụ thể; chưa đủ để công bố độ chính xác phát hiện trên lưu lượng doanh nghiệp hoặc tỷ lệ false positive có giá trị tổng quát.

Suricata đã tạo một số cảnh báo trong lúc hoạt động quản trị SSH/HTTP hợp lệ. Báo cáo ghi nhận mức ước tính 8–12% trong quá trình quan sát, nhưng chưa có tổng mẫu, quy trình gắn nhãn và ma trận nhầm lẫn đầy đủ. Do đó con số này chỉ gợi ý nhu cầu tinh chỉnh luật phù hợp lưu lượng nội bộ, không được xem là chỉ số chất lượng chính thức.

Isolation Forest phát hiện được các spike CPU/RAM và lưu lượng nhân tạo đã thử, xác nhận chuỗi huấn luyện–suy luận–cảnh báo. Tuy nhiên, `contamination=0.05` là tham số thiết kế chứ không phải xác suất báo động sai đo được; chưa có phép thử với tập dữ liệu độc lập để khái quát khả năng phát hiện hay tỷ lệ bỏ sót.

CHƯƠNG 5. ĐÁNH GIÁ TỔNG THỂ VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả chính và giá trị của dự án

5.1.1 Các chức năng đã triển khai và kiểm chứng

Hệ thống đã tích hợp các chức năng cần thiết cho một phòng lab giám sát: SNMP/Node Exporter qua Prometheus, thu thập sự kiện Suricata từ các vùng mạng, alert theo ngưỡng và vòng đời, Isolation Forest trên các chỉ số mục tiêu, cùng giao diện Flask, Telegram và AI Agent. Kết quả xác nhận các đường xử lý chính hoạt động trong những kịch bản đã thử, không đồng nghĩa sản phẩm đã được đánh giá đầy đủ về độ tin cậy hoặc bảo mật trong môi trường sản xuất.

Topology PNETLab với sáu thiết bị mục tiêu cho phép kiểm tra các tình huống mất liên kết, tăng tải và lưu lượng an ninh có kiểm soát. Bảng tổng hợp Chương 4 ghi nhận 13/13 kịch bản chính đạt kỳ vọng trong phạm vi lab; kết quả này không tự chứng minh độ chính xác trên các loại tấn công chưa kiểm thử. Thời gian phát hiện thay đổi theo từng nguồn dữ liệu và quy tắc kích hoạt, chưa phải chỉ tiêu SLA của doanh nghiệp.

Kiến trúc phân lớp và Repository Pattern giúp tập trung truy vấn dữ liệu; cấu hình thiết bị và ngưỡng được tổ chức để dễ bảo trì. Tuy nhiên, khả năng mở rộng thực tế phải được xác minh bằng thử nghiệm tăng số thiết bị, tải API, thời gian lưu trữ và xử lý nhiều sự kiện đồng thời. Đây là ưu điểm thiết kế của mã nguồn, không phải năng lực đã được benchmark ở quy mô lớn.

5.1.2 Kinh nghiệm thiết kế hệ thống giám sát tích hợp

Bài học quan trọng nhất là kiểm thử theo đường đi dữ liệu: thiết bị tạo giá trị, exporter hoặc IDS ghi nhận, nơi lưu trữ cập nhật, engine phát hiện đánh giá và dashboard/Telegram thể hiện kết quả. Mỗi bước có thể hỏng độc lập. Giao diện trực quan không thay thế xác minh metric hoặc EVE JSON; một sự kiện IDS cũng cần được gắn đúng nguồn và thời điểm để có giá trị điều tra.

Dự án cũng nhấn mạnh ranh giới giữa dấu hiệu quan sát và kết luận. Interface có lưu lượng không bảo đảm một dịch vụ ứng dụng hoạt động; Suricata khớp chữ ký không chứng minh khai thác thành công; Isolation Forest nhận ra ngoại lai không xác định

động cơ tạo lưu lượng. Quy trình xử lý sự cố đáng tin cậy cần kết hợp log, số liệu và các phép kiểm tra tại nguồn trước khi ra quyết định.

5.2 Giới hạn kỹ thuật và vận hành

Đánh giá dự án cần phân biệt "chạy được trong lab" với "sẵn sàng vận hành chính thức". Topology sáu thiết bị phù hợp kiểm thử nguyên lý nhưng chưa phản ánh phân tán hạ tầng, các luồng dữ liệu quy mô lớn hoặc yêu cầu tuân thủ tại môi trường sản xuất. Các hạn chế dưới đây định hướng cho kế hoạch nâng cấp thay vì phủ nhận kết quả đã đạt được.

5.2.1 Môi trường thử nghiệm và dữ liệu học máy

Mô hình chạy trên nền tảng giả lập PNETLab, chưa có phép thử kéo dài trên nhiều máy vật lý hoặc mạng sản xuất. Kết quả về độ trễ, sử dụng CPU/RAM và tính ổn định chịu ảnh hưởng của tài nguyên host; chúng chưa thể dùng để cam kết hành vi của hệ thống khi giám sát số lượng thiết bị lớn hơn.

Baseline học máy khoảng bảy ngày chưa bao quát đầy đủ chu kỳ sử dụng theo tuần, tháng hoặc dịp đặc biệt. Khả năng cảnh báo sai có thể tăng khi môi trường thay đổi dù không xuất hiện tấn công. Cần mở rộng dữ liệu, gắn nhãn các tình huống kiểm thử và đánh giá theo từng loại metric trước khi thay đổi mô hình.

5.2.2 Bảo mật ứng dụng và lưu trữ dài hạn

Ứng dụng web chưa có xác thực và phân quyền người dùng hoàn chỉnh. Dù các khóa dịch vụ được đặt trong biến môi trường, việc truy cập dashboard và API trên mạng thực cần bổ sung TLS, quản lý phiên, quyền tối thiểu và nhật ký truy cập; đây là điều kiện ưu tiên trước khi công bố hệ thống cho người dùng ngoài phòng lab.

AI Agent hỗ trợ giải thích dữ liệu và gợi ý điều tra, không có cơ chế tự thực thi lệnh khắc phục. Đây là giới hạn có chủ đích về an toàn: mọi thay đổi firewall hoặc cấu hình thiết bị cần được xác nhận theo quy trình kiểm soát thay đổi trước khi áp dụng.

Dữ liệu cảnh báo và sự kiện an ninh tăng theo thời gian; phiên bản thử nghiệm chưa có kế hoạch retention, rotation, archiving và phục hồi dài hạn được kiểm chứng. Giới

hạn số bản ghi được hiển thị hoặc xử lý không thay thế chiến lược lưu trữ an toàn cho PostgreSQL và log Suricata.

5.3 Lộ trình hoàn thiện

Lộ trình hợp lý là ưu tiên tính an toàn và khả năng tái lập trước các tính năng nâng cao: triển khai xác thực/TLS, hoàn thiện lưu trữ và sao lưu, đo lường với dữ liệu gắn nhãn, rồi mới xem xét tự động hóa phản ứng. Mỗi bước cần có tiêu chí nghiệm thu và phương án hoàn tác phù hợp với rủi ro của chức năng.

5.3.1 Mở rộng giám sát, bảo mật và cơ chế phản ứng

Mở rộng nguồn dữ liệu có thể bắt đầu với NetFlow/sFlow để phân tích lưu lượng theo luồng, cùng các exporter ứng dụng dành cho Nginx và PostgreSQL. Mục tiêu là tăng độ bao phủ nhưng cần cân đối chi phí thu thập, số lượng nhãn và thời gian giữ dữ liệu; không nên chỉ tăng tần suất scrape khi chưa đánh giá tài nguyên.

Các phản ứng tự động như chặn IP hoặc truy vấn tiến trình có thể được nghiên cứu trong môi trường cô lập với cơ chế phê duyệt thủ công, danh sách cho phép, giới hạn thời gian và rollback. Không nên cấp quyền điều khiển trực tiếp cho mô hình ngôn ngữ chỉ dựa trên một alert, vì dấu hiệu quan sát được có thể là cảnh báo sai.

Đối với giao diện và API, cần bổ sung xác thực người dùng, phân quyền theo vai trò, HTTPS và rate limiting; đồng thời quản lý token/SSH key an toàn, lọc dữ liệu đầu vào và lưu dấu truy cập. Đây là điều kiện tiền đề cho mọi kế hoạch cung cấp dịch vụ ngoài lab.

5.3.2 Nâng cao phát hiện bất thường và trải nghiệm vận hành

Sau khi có dữ liệu lịch sử và nhãn đủ tin cậy, nhóm có thể so sánh Isolation Forest với các phương pháp có xét tính chu kỳ, bao gồm mô hình chuỗi thời gian. Việc đánh giá cần báo cáo precision, recall, false positive rate và độ trễ theo từng loại bất thường, thay vì chọn mô hình chỉ vì phát hiện được các spike rất lớn.

Phần giao diện có thể phát triển theo hướng báo cáo vận hành định kỳ, bộ lọc điều tra thuận tiện hơn và trải nghiệm trên thiết bị di động. Mọi tính năng mở rộng cần giữ khả

năng truy vết nguồn dữ liệu và trạng thái xử lý để người nhận cảnh báo không hiểu nhầm gợi ý của AI thành xác nhận sự cố.

Thứ tự triển khai đề xuất gồm: chuẩn hóa nguồn dữ liệu → kiểm tra bảo mật và lưu trữ → xây dựng bộ benchmark gắn nhãn → đánh giá mô hình → thử nghiệm phản ứng bán tự động. Hướng này giúp nâng chất lượng phát hiện mà không đánh đổi tính an toàn của hạ tầng để lấy thêm tính năng trình diễn.

KẾT LUẬN

NetMon SOC đã kết hợp được dữ liệu trạng thái thiết bị từ SNMP/Node Exporter, sự kiện Suricata, kiểm tra dịch vụ và điểm ngoại lai của Isolation Forest vào một quy trình quan sát. Ứng dụng Flask/PostgreSQL tổ chức bản ghi và giao diện; Telegram chuyển thông báo đến người phụ trách; AI Agent hỗ trợ diễn giải ngữ cảnh. Kiến trúc này hữu ích để thực hành kiểm tra nguyên nhân và đối chiếu tín hiệu của hệ thống mạng.

Theo tài liệu kiểm thử, 13 kịch bản chính đạt kết quả kỳ vọng trong phạm vi mô hình PNETLab. Kết luận đó không đồng nghĩa hệ thống đã được kiểm định trên mạng sản xuất, có độ chính xác phát hiện xác định hay khả năng chịu tải dài hạn. Giá trị của dự án nằm ở khả năng tái lập từng bước và các giới hạn được chỉ rõ, tạo cơ sở nâng cấp theo hướng bảo mật truy cập, đánh giá dữ liệu có nhãn và quản lý vận hành chặt chẽ hơn.

TÀI LIỆU THAM KHẢO

[1] Bejtlich, R. (2004). The Tao of Network Security Monitoring: Beyond Intrusion Detection. Addison-Wesley Professional, Boston, MA. Truy cập tại: https://www.pearson.com/en-us/subject-catalog/p/tao-of-network-security-monitoring-the-beyond-intrusion-detection/P200000009342

[2] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining, 413–422. IEEE. Truy cập tại: https://ieeexplore.ieee.org/document/4781136

[3] Prometheus Authors. (2023). Prometheus Documentation - Querying and Storage. Prometheus.io. Truy cập tại: https://prometheus.io/docs/introduction/overview/

[4] OISF - Open Information Security Foundation. (2023). Suricata User Guide. Truy cập tại: https://docs.suricata.io/en/latest/

[5] Nguyễn Gia Thọ và cộng sự. (2021). Giáo trình An ninh mạng và Bảo mật thông tin. Nhà xuất bản Đại học Quốc gia Hà Nội, Hà Nội. Truy cập tại: https://repository.vnu.edu.vn/handle/VNU_123/144256

[6] Trung tâm Ứng cứu khẩn cấp không gian mạng Việt Nam - VNCERT/CC. (2023). Hướng dẫn xây dựng Trung tâm điều hành an ninh mạng (SOC) cho cơ quan, tổ chức. Cục An toàn thông tin, Bộ Thông tin và Truyền thông. Truy cập tại: https://ais.gov.vn/huong-dan-xay-dung-soc